future internet

MDPI

Review

AI Agent Communications in the Future Internet—Paving a Path Toward the Agentic Web

Qiang Duan [1,*] and Zhihui Lu [2]

[1] Information Sciences & Technology Department, Pennsylvania State University, Abington, PA 19001, USA
[2] College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200438, China; lzh@fudan.edu.cn
* Correspondence: qduan@psu.edu

Abstract

The rapid evolution of artificial intelligence technologies toward the agentic AI paradigm enables the emergence of the Agentic Web in the future Internet. Agent communication plays a critical role in constructing the Agentic Web but faces unique challenges posed by the edge–network–cloud continuum in the future Internet. This paper provides a comprehensive overview of state-of-the-art agent communication protocols and technologies, evaluating their readiness to support the construction of the Agentic Web. We first survey representative communication protocols and analyze the key technologies they employ, assessing their effectiveness in addressing the challenges for agent communications in the future Internet. We then identify critical gaps between existing approaches and the requirements of the Agentic Web, and propose a unified architectural framework grounded in virtualization and service-oriented principles to address these gaps. Such a framework may greatly facilitate the development of a pluralistic ecosystem in which various agent communication technologies and protocols can be freely developed and fully utilized. We also discuss open topics and possible directions for future research toward a fully realized Agentic Web.

Keywords: agentic AI; multi-Agent Systems; agent communications; Agentic Web; the future Internet

1. Introduction

The recent evolution of artificial intelligence is moving toward an agentic AI paradigm, in which AI agents can autonomously perceive their environment, make decisions, take actions, and interact with one another [1–4]. Multi-agent interactions are expected to be a key attribute of the emerging agentic AI paradigm, and Multi-Agent Systems (MASs) comprising agents that interact, either cooperatively or competitively, form a foundation for realizing agentic AI [5–7]. Furthermore, recent developments in lightweight AI technologies, such as model compression and pruning [8,9], together with specialized hardware for AI acceleration, allow the wide deployment of MAS-based agentic AI applications in edge computing environments [10].

In parallel with the rapid progress of agentic AI, recent developments in networking technologies, including network virtualization, software-defined networking, and network-as-a-service, enable the convergence of networking and edge–cloud computing into an edge–network–cloud continuum, thereby transforming the Internet from a data transport system into an infrastructure for integrated communication and computation [11]. Such a

Academic Editor: Ivan Serina

Received: 1 February 2026
Revised: 10 March 2026
Accepted: 16 March 2026
Published: 21 March 2026

composite network-compute infrastructure in the future Internet enables the widespread deployment of MASs across various scenarios, ranging from cloud-based data centers to edge-based Internet of Things (IoT) systems, and from wireline backbone networks to wireless access networks. Therefore, we envision numerous MASs for diverse agentic AI applications distributed across the edge–network–cloud continuum in the future Internet, forming an *Agentic Web*—a web of AI agents interconnected through the future Internet's infrastructure.

Agent communications, the mechanisms that enable information exchange among autonomous AI agents, play a crucial role in constructing effective MASs [12]. A main obstacle to effective agent interactions in MASs is the lack of standardized communication protocols to ensure interoperability among autonomous AI agents, which could be developed by different vendors, implemented with heterogeneous technologies, configured for diverse roles, and deployed by numerous organizations. Agent communications have recently emerged as an active research area attracting extensive attention from both academia and industry, and exciting progress has been made in this field [13]. A variety of agent communication protocols have been proposed over the past two years, including the Agent2Agent (A2A) protocol, the Agent Communication Protocol (ACP), and Agent Network Protocols. In addition, various new developments in agent protocols are currently ongoing.

The construction of Agentic Web, which interconnects numerous AI agents in the future Internet, introduces unique challenges for agent communication. Unlike conventional multi-agent collaboration, the notion of Agentic Web highlights the integration of MASs and Internet infrastructure, in which a massive number of heterogeneous AI agents, comprising various MASs operating in parallel for diverse applications, are deployed across a highly dynamic, large-scale edge–network–cloud continuum. Although they have shown encouraging performance in various MAS scenarios, the currently available agent communication protocols are often designed for specific networking environments rather than explicitly for the Agentic Web in the future Internet. Therefore, agent communications in the future Internet for constructing the agentic Web is a critical, exciting, and challenging research problem that must be fully resolved to realize the notion of agentic AI.

A variety of papers have been published to review the developments in the technologies and protocols for agent communications; for example, the surveys presented in [13–16]. However, these works either focus on a particular protocol, e.g., A2A as in [14,15], overlook some projects related to agent communications, e.g., LMOS and AGNTCY missed in [16,17], or lack reflection of the latest progress in this field, e.g., the merger of A2A and ACP protocols. Also, none of these surveys particularly assesses agent communication protocol designs against the specific demands of Agentic Web in the future Internet.

In this paper, we aim to provide a comprehensive overview of the state of the art in agent communication technologies with respect to their readiness to address the unique challenges of constructing the Agentic Web in the future Internet. Specifically, in this paper, we discuss the key functionalities of agent communication for constructing an Agentic Web, identify the main challenges to agent communication for Agentic Web introduced by the edge–network–cloud continuum in the future Internet, present a survey of the representative protocols for agent communications, and assess key technologies leveraged in these protocols in terms of their effectiveness in addressing the challenges to agent communications in the future Internet. Based on the survey and assessment, we analyze the gap between current agent communication protocols and the Agentic Web objective, and propose a unified architectural framework for the Agentic Web grounded in virtualization and service-oriented principles to address these gaps. This framework may greatly facilitate a pluralistic ecosystem in which various agent communication technologies and protocols can be freely developed and fully utilized. We also discuss key topics and potential

directions for future research to realize this framework and fully realize the notion of the Agentic Web.

2. AI Agent Communications for Agentic Web

2.1. From Multi-Agent Systems to Agentic Web

The rapid development of agentic AI over the past few years has followed a trajectory from individual AI agents to homogeneous multi-agent systems, and then to heterogeneous multi-agent systems. Early efforts of large model-driven MAS, e.g., Refs. [18–20], mainly focus on task division and collaboration among AI agents with different roles and skills and are orchestrated by frameworks like AutoGen [21], role-based CrewAI [22], or state-machine-driven LangGraph [23]. These MASs typically assume that the AI agents involved are deployed and operated within the same organization and hosted on a homogeneous infrastructure platform (e.g., in the same cloud data center). The current trend in MAS development is the transition to heterogeneous multi-agent systems, which move beyond the single-vendor model to include agents implemented by different developers using diverse technologies, deployed across multiple organizational domains, and hosted on heterogeneous platforms, ranging from cloud servers to edge computing devices, that are connected via the Internet [24].

The trend toward heterogeneous MASs leads to the formation of the Agentic Web—an Internet of AI agents built on the converged network and cloud–edge computing infrastructure [11]. Unlike conventional MASs, the notion of Agentic Web presents a holistic vision of heterogeneous multi-agent systems and Internet infrastructure, highlighting cross-organizational deployment, Internet-scale distribution, extreme heterogeneity in both AI agents and their hosting platforms, and close integration between MASs and edge–network–cloud infrastructures. Cloud data centers typically comprise a cluster of homogeneous servers that provide high computational capacity, whereas edge computing systems often consist of heterogeneous, resource-constrained hosts, such as IoT devices. The networking systems that connect data centers and IoT devices span fixed backbone networks and wireless mobile access networks. Therefore, the edge–network–cloud continuum of the Internet infrastructure brings in various new challenges to agent communications in the Agentic Web.

2.2. An Illustrative Scenario of Agentic Web in the Internet

As an illustrative scenario of Agentic Web in the Internet, Figure 1 shows a smart city environment in which AI assistant agents are deployed across a massive number of user devices at the network edge, including smartphones on cellular networks, laptops on Wi-Fi networks, PCs on home networks, and automobile computers on vehicular networks. In addition, industry/enterprise-level AI agents from different organizations, serving various functions, are hosted on edge servers and in cloud data centers. These AI agents are connected via the Internet infrastructure to form an Agentic Web upon which various multi-agent systems can be deployed for different applications.

For example, a multi-agent system is built for a cooperative Smart Transport and Healthcare (STH) application. This system comprises AI agents deployed on ambulances for automatic routing and navigation, agents hosted on edge servers (e.g., roadside units) to control traffic signals based on traffic monitoring, and agents in the healthcare system's data centers to allocate resources across the city's medical facilities. These AI agents collaborate to coordinate autonomous driving, smart transportation, and emergency response, allowing the ambulance to autonomously travel through an optimal route to the nearest facility with the required treatment capacity.

Another example of a multi-agent system deployed on the Agentic Web is for an open Collaborative Research Community (CRC). In this system, hundreds of agents voluntarily participate in a community to conduct open, collaborative research to investigate how people's activities affect the city's environment. The participating agents play different roles, including sensor agents that collect data, research agents that analyze data and propose models, actuator agents that conduct experiments, and criticizer agents that provide feedback. Among the agents, sensors could be deployed on resource-constrained edge devices scattered across the city, whereas researcher agents may be hosted on cloud servers within enterprise networks.

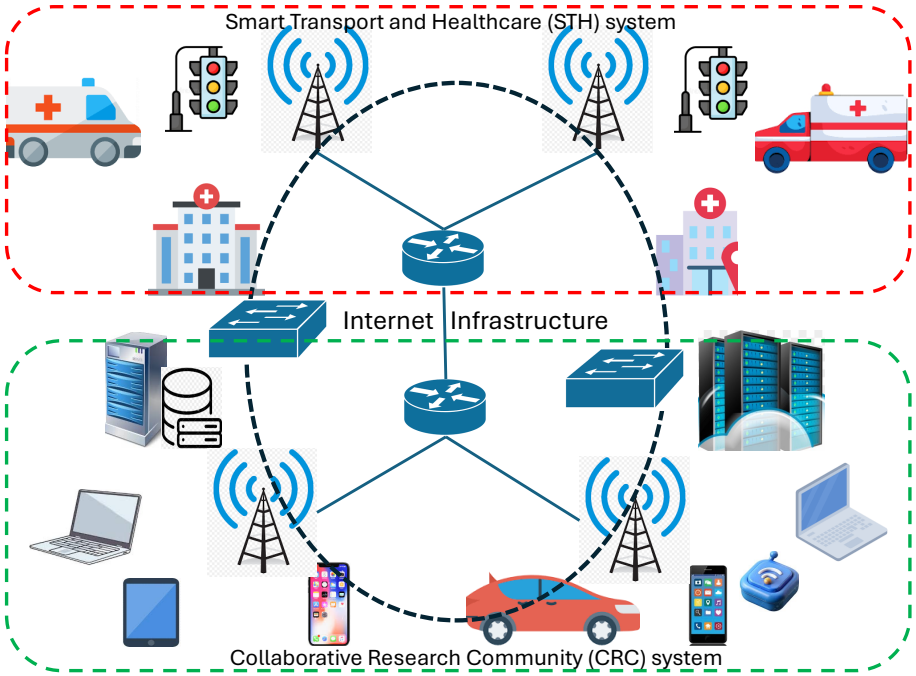

Figure 1. An illustrative scenario for agent communications in Agentic Web, with two use cases—Smart Transport and Healthcare (STH) and Collaborative Research Community (CRC).

2.3. The Role of Agent Communications in Agentic Web

Agent communications enable interactions among autonomous AI agents to support the intricate dynamics in MAS. An agent communication protocol leverages the computational and communication resources in the Internet infrastructure to build an information-exchange platform that supports interactions among AI agents in a MAS. Therefore, agent communication plays a crucial role in realizing the Agentic Web and thus significantly impacts the performance of Agentic AI applications in the future Internet.

Figure 2 presents a layered architecture that illustrates the role of agent communications in the Agentic Web deployed on the Internet [15]. In this architecture, the agent communication layer sits between the upper-layer multi-agent systems built for various Agentic AI applications and the underlying Internet infrastructure comprising an edge–network–cloud continuum. The core functionalities of the agent communication layer can be categorized into two groups that are respectively for (i) management of the inter-agent communication systems and (ii) transportation for information exchange between AI agents. It is worth noting that the functions in these two categories, though focusing on distinct aspects of agent communication systems, are closely related. For example, virtually all the management functions rely on the information transport capabilities to accomplish their goals, while the operations of information transport functions are governed by the management functions.

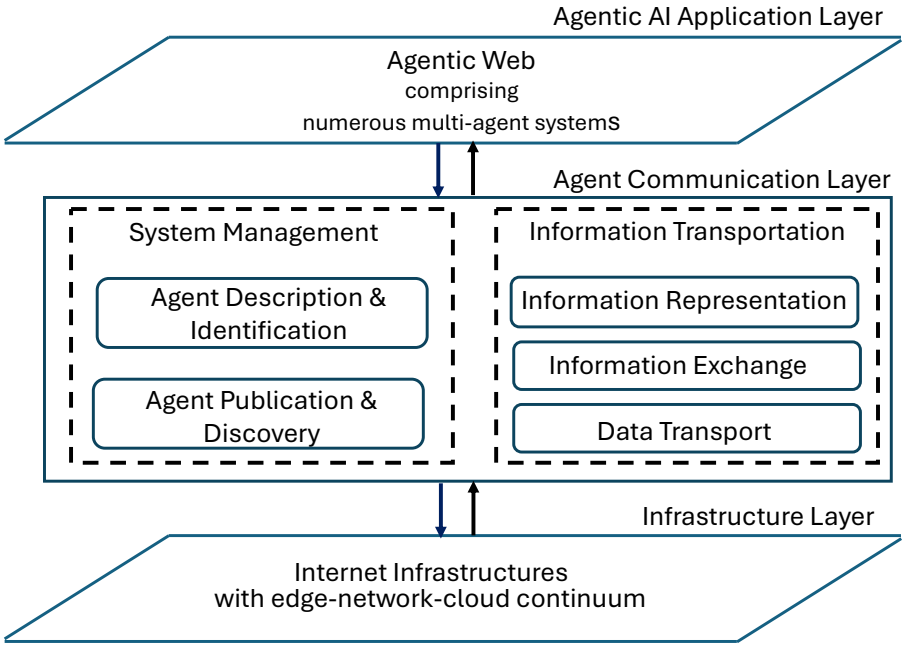

Figure 2. The role and functions of agent communications for Agentic Web in the future Internet.

2.4. Key Functionalities of Agent Communications for Agentic Web

System Management Functions: The system management functions govern the description, publication, discovery, identification, and authentication of the AI agents in the communication system. The agent description function articulates an agent's capabilities and services in a machine-readable, readily comprehensible format for other agents. Agent publication and discovery functions ensure each agent's description is accessible to other agents, allowing them to locate available agents and select the appropriate one(s) to communicate with to accomplish specific tasks. Agent identification and authentication functions assign each agent a unique identity and rigorously verify it. The system management aspect of an agent communication protocol is also expected to provide support for upper-layer MAS management, typically for agent-orchestration functionality.

Information Transportation Functions: Transportation for information exchange refers to the actual transmission of information between AI agents. The functions in this category include information representation, information exchange, and data transport. Information representation dictates how exchanged information is presented in a standard format, with options including structured formats such as JSON and unstructured formats such as natural language, which can be understood by heterogeneous agents. The information exchange function controls message exchange patterns, such as synchronous versus asynchronous messaging, between AI agents. The data transport function ensures the delivery of inter-agent messages across a network, including sub-functions such as sending, forwarding, routing, and receiving.

2.5. Agent Communication System in Agentic Web

The overall structure of the agent communication system in a simple example MAS and its operation process are illustrated in Figure 3. The agent protocol operated by the system involves the following actors in the communications process. A *user*, which can be a human or a software application, invokes a request (intention) to the MAS to fulfill a specific need. A *client agent*, an AI agent in the MAS with which the user has direct access, translates the user's intent into requests for specific tasks that can be delegated to other agents. Then the client agent interacts with a *system controller*, which can be implemented as a central server or via a decentralized cooperation mechanism, to discover and select the appropriate

remote agents that meet the task requirements. The agent communication protocol also specifies standards for agent descriptions and provides a mechanism for disseminating them. After agent discovery, the client agent begins operating communication sessions with the selected remote agents, including representing information in formats the remote agents can understand and managing message exchange patterns. These communication sessions utilize the computing and networking resources in the Internet infrastructure to ensure data transport between agents. Each remote agent is responsible for processing the delegated tasks and providing results to the client agent, which then renders the returned results into the final outcome and responds to the user.

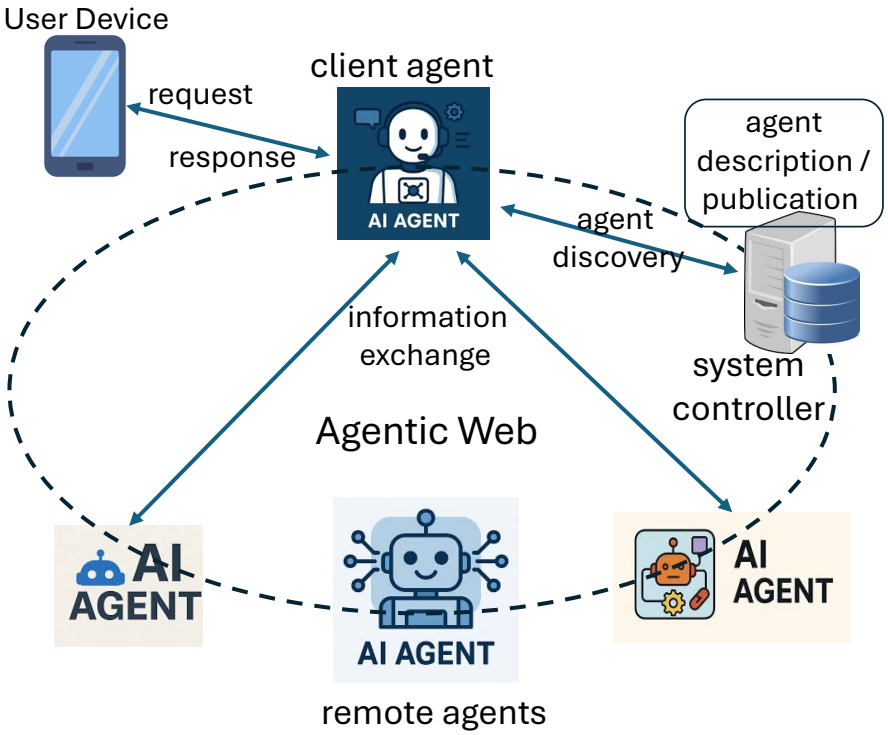

Figure 3. The agent communication system and its operation process in Agentic Web.

2.6. Challenges to AI Agent Communications in Future Internet

Deploying MAS across an edge–network–cloud continuum to build an Agentic Web in the future Internet introduces unique challenges related to heterogeneity, scalability, dynamism, and efficiency that stress agent communication technologies and protocols.

2.6.1. The Challenge of System Heterogeneity

The Agentic Web in the future Internet is inherently a highly heterogeneous system. It comprises a wide range of AI agents with diverse skills, roles, and configurations. These agents are developed by different vendors, operated in different organizations, and tailored toward different application objectives. Furthermore, the AI agents in Agentic Web can be deployed across hosts with heterogeneous implementation technologies and various computing capacities, ranging from battery-powered edge devices to cloud servers in data centers, and connected to a variety of network environments, from wireless mobile networks to high-speed data center networks. Therefore, agent communication to achieve interoperability among diverse AI agents deployed across the Internet infrastructure, using heterogeneous networking and computing technologies, is particularly challenging.

2.6.2. The Challenge of Massive Scalability

The Agentic Web is expected to comprise a large number of agents, forming numerous MASs hosted on a vast array of edge/cloud devices distributed across large-scale Internet infrastructure, to serve a massive group of customers running various agentic AI applications. Information exchange among a large number of AI agents deployed across the Internet may generate a huge volume of communication traffic that overwhelms bandwidth-constrained networks, especially at the Internet edge. Managing and coordinating communications among numerous, geographically dispersed agents to accommodate potentially large volumes of information exchange presents significant scalability hurdles.

2.6.3. The Challenge of Environmental Dynamicity

Agentic Web in the future Internet will form a highly dynamic environment for inter-agent communication. The multi-agent interactions in the Agentic Web are expected to be dynamic, with individual agents regularly joining or leaving a MAS and cooperation patterns among agents evolving. The edge–network–cloud continuum in the Internet, which provides the infrastructure platform for the Agentic Web, is also highly dynamic. The availability of computational and communication resources in the infrastructure varies over time; for example, edge devices may be switched on and off to save energy, while network bandwidth may fluctuate due to changes in transmission link states. Furthermore, the mobility of both mobile devices (as agent users) and edge devices (as agent hosts) introduces additional dynamism to agent communications.

2.6.4. The Challenge of Resource Constraints

The infrastructure for Agentic Web in the future Internet is built on the edge–network–cloud continuum, where computational and communication resources may be constrained, especially at the edge. Edge devices hosting mobile agents often have restricted processing power, storage space, and battery capacities. Network connections among edge devices often rely on wireless channels with limited bandwidth. On the other hand, the LLMs that underpin modern AI agents are particularly resource-intensive and often exceed the capacity of most edge devices. Multi-modal agentic AI applications that process multimedia information may also trigger high-volume data transmission among collaborative agents. Therefore, resource constraints in Internet infrastructure and resource-demanding agentic AI applications together pose a severe challenge to the efficiency of agent communication in the future Internet.

2.6.5. The Challenge of Security

The highly distributed deployment of multi-agent systems in the edge–cloud environment introduces new security vulnerabilities and threats that exceed the scope of traditional network security [25,26]. AI agents operate with high autonomy and often have broad access to sensitive tools and data, serving as "digital insiders" within enterprise systems. This autonomy expands the attack surface to include semantic-layer vulnerabilities, such as prompt injection and goal hijacking, in which malicious inputs can manipulate an agent's reasoning process. Furthermore, the decentralized nature of the Agentic Web necessitates a trade-off between low-latency coordination and strict verification, often leading to implicit trust vulnerabilities in which a compromised edge agent can trigger a cascading failure across the entire collective. Therefore, ensuring secure agent communications in the future Internet becomes a critical challenge.

2.6.6. Intertwined Challenges to Agent Communications in Future Internet

The aforementioned challenges to agent communications in the future Internet are deeply and subtly intertwined. For example, environmental dynamicity requires frequent control actions and updates (e.g., to rediscover the optimal agents and adjust communication patterns). However, in a large-scale system with limited resources in some parts, this constant churn can create a signaling storm that consumes the very network and computational resources that are already severely constrained. Furthermore, the system's heterogeneity makes it even harder to design uniform policies to manage this dynamic, resource-constrained environment. Therefore, any viable agent communication technology for Agentic Web in the future Internet must be designed to address this entire complex system of interacting constraints, rather than a single challenge in isolation.

3. Representative Agent Communication Protocols

3.1. A Landscape of Agent Communication Protocols

Research on communication technologies for AI agents has recently gained momentum, resulting in a variety of protocol designs. Although excited by the rapid progress, researchers who have just entered this emerging field might find the broad range of protocols developed by various organizations, each focusing on different aspects of agent communication systems, overwhelming or even confusing. In this section, we present a landscape of current agent protocol developments, providing readers with a broad overview of the field's current status.

The problem of communication among autonomous agents has been studied in various contexts. Early foundational protocols include the FIPA-ACL (Foundation for Intelligent Physical Agents—Agent Communication Language), which provides a standard speech-act-based language for agent communication. Various approaches to agent communication have been developed in multi-agent reinforcement learning, and a comprehensive survey of related technologies is presented in [27]. However, these communication approaches were not specifically designed for systems consisting of LLM-driven AI agents and thus fall outside the scope of our review.

In this paper, we focus on the recently developed protocols for agent communications in the agentic AI paradigm. This field has gained momentum since November 2024, when Anthropic announced its Model Context Protocol (MCP) [28]. In 2025, multiple organizations announced development projects for various protocols. The AGNTCY and LMOS projects were announced in March by Cisco's Outshift and the Eclipse Foundation, respectively. The Agent Communication Protocol (ACP) was published by IBM Research in March, followed by the Agent2Agent (A2A) protocol announced by Google in April. The open-source community has also been actively developing protocols for agent communication since late 2024, and the ANP-Community published its Agent Network Protocol (ANP) as a W3C White Paper in May 2025 [29]. The objective of our review in this paper is not to be exhaustive of all protocols under development, but to select the most representative ones as of the time of writing (early 2026) to provide readers with a broad overview of the current landscape of this rapidly changing field.

As depicted in Figure 4, the current representative agent protocols can be categorized based on their main design objectives into three groups: (i) inter-agent protocols for controlling communications among autonomous AI agents, (ii) context-oriented protocols, represented by the MCP [28], designed for communications between an AI agent and its context, typically for using tools and/or accessing data sources; and (iii) user-oriented protocols, for example the Agent-User Interaction protocol (AG-UI) [30], focusing on interactions between an agent and front-end applications. Protocols in these three categories complement each other and are all required in a MAS; on the other hand, inter-agent

communication protocols play a key role in realizing the Agentic Web due to their explicit focus on multi-agent interactions.

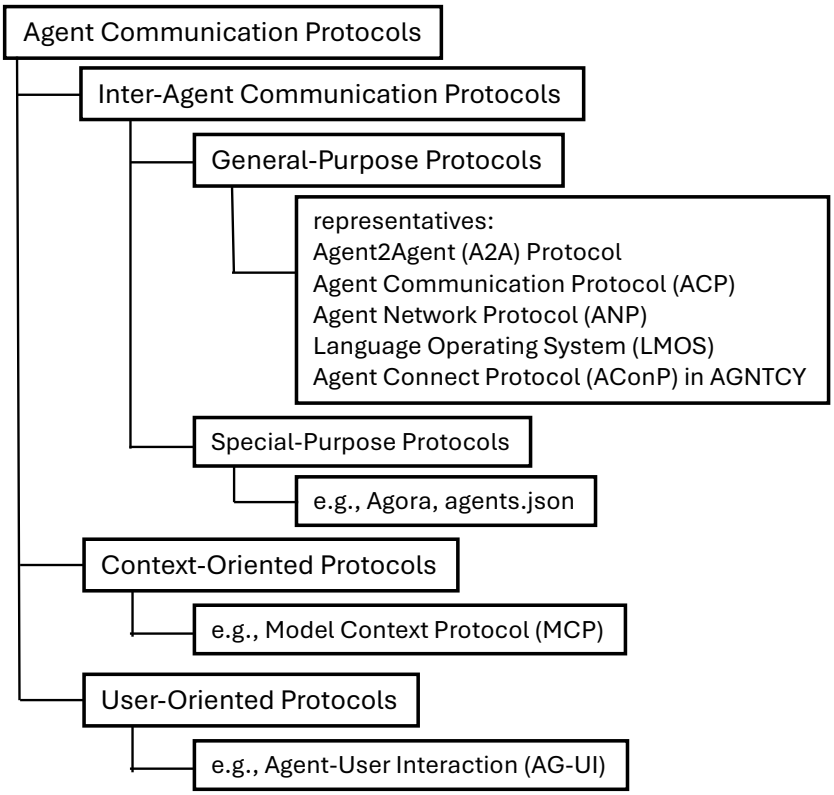

Figure 4. A design objective-based taxonomy of representative agent communication protocols.

The protocols for inter-agent communication can be further divided into general-purpose and special-purpose protocols. Each general-purpose protocol provides a full stack of functions for both system management and information transport in general MAS scenarios, while special-purpose protocols focus on specific aspects of inter-agent communication. Representative general-purpose agent protocols include the Agent2Agent Protocol (A2A) [31], Agent Communication Protocol (ACP) [32], Agent Network Protocol (ANP) [33], Language Model Operating System (LMOS) [34] from the Eclipse Foundation, and Agent Connect Protocol (AConP) designed in the AGNTCY project [35]. Examples of special-purpose protocols include the Agora protocol [36] for negotiation of communication schemes between AI agents and agents.json [37] for agent description and discovery. Since general-purpose protocols provide comprehensive solutions to inter-agent communication with mechanisms for both system management and information transportation, the functionalities of special-purpose protocols naturally overlap with parts of those of general-purpose protocols. The survey presented in this article focuses on comprehensive, general-purpose protocols for inter-agent communications.

Our main criteria for selecting representative general-purpose protocols for inter-agent communication are their ability to provide all key functions for both system management and information transportation, as listed in Section 2.4. It is worth noting that among the selected protocols, LMOS provides an operating system framework for standardizing how LLMs and AI agents interact with the underlying infrastructure, and the AGNTCY project specifies a full-stack infrastructure for multi-agent systems; both offer more comprehensive solutions to multi-agent collaboration than merely protocols for agent communication. Our survey and analysis of LMOS and AGNTCY in this paper focus on the agent communication-related mechanisms provided by these two solutions.

3.2. Agent2Agent Protocol (A2A)

The Agent2Agent (A2A) protocol [31] was officially announced by Google in April 2025 and then donated to the Linux Foundation in June 2025 to establish it as an open-source, community-governed project. It is arguably the most widely adopted industry-backed protocol for inter-agent communications as of the end of 2025 [15]. It is explicitly designed to allow agents from different vendors, built on diverse frameworks, to collaborate securely across enterprise environments. To achieve this objective, the protocol employs a client-server model based on established enterprise standards and follows a principle of "opaque execution" to enable agent collaboration without sharing their internal memory, prompts, or proprietary tool implementations [14].

In the A2A protocol architecture, client agents initiate requests and delegate tasks to one or more remote agents, which expose their access endpoints, accept tasks, process them, and return results. The Agent description in A2A uses the *Agent Card*, a structured JSON [38] file that typically contains metadata about an agent, including its name/identifier, a summary of its functions or "skills", and the endpoint URL for accessing the agent. The A2A protocol supports multiple schemes for agent publication/discovery. The open discovery method allows agents to make Agent Cards accessible via a standard "well-known" path https://agent-domain/.well-known/agent.json. The registry-based approach publishes Agent Cards on a central server, enabling available clients to be discovered. The API-based discovery approach limits agents to offering customized API endpoints and delivering Agent Cards only to authenticated clients.

The A2A protocol employs a structured approach for information representation, primarily utilizing JSON-RPC 2.0 as the standardized data exchange format for requests and responses. The fundamental unit of work is a *Task*, which has a unique ID and a defined lifecycle (e.g., submitted, working, completed, failed). The tangible output generated by the remote agent (e.g., a file, a block of text, structured JSON data) is referred to as an *Artifact*. The A2A protocol supports both synchronous and asynchronous message exchange between agents. In the synchronous mode, the client agent sends a request and waits for a direct response from the remote agent. A2A provides two asynchronous messaging schemes for long-running processes. Server-Sent Events (SSEs) [39] are used to stream updates from a remote agent to the client in real time. A client agent can also register webhooks to receive asynchronous push notifications whenever the task's status changes.

3.3. Agent Communication Protocol (ACP)

The Agent Communication Protocol (ACP) was initially developed by IBM Research to power its open-source BeeAI Platform [32]. Soon after announcing ACP in March 2025, IBM donated the BeeAI project, including the ACP specification, to the Linux Foundation. In August 2025, ACP merged into the A2A protocol under the Linux Foundation's governance. In this section, we briefly review the key features of the original ACP design, which addresses some of A2A's limitations and is particularly tailored for resource-constrained scenarios, such as edge computing systems.

The ACP protocol follows a different architectural philosophy than A2A. While A2A is based on JSON-RPC, ACP was designed as a REST-native protocol. It uses simple, well-defined REST endpoints and standard HTTP patterns (GET, PUT, POST, etc.) for all interactions [40]. The simplicity that comes from avoiding RPC and adopting RESTful interactions often makes ACP more lightweight, integration-friendly, and better aligned with modern edge-native development.

The ACP design achieves modality-agnosticity by using standard HTTP MIME types (e.g., application/json, image/png) to identify content, making the protocol inherently extensible to any data format (text, images, video) without requiring protocol changes. The

protocol was designed primarily for asynchronous communications, ideal for long-running agent tasks, while still supporting synchronous requests.

Offline agent discovery is arguably ACP's most significant contribution. The *Agent Manifest* (ACP's equivalent of the Agent Card) was designed to support offline discovery by embedding agent metadata directly in its distribution package (e.g., as labels on a container image). This feature enables agent discovery in scenarios where an agent may be inactive (and thus not serving its manifest) until needed, thereby better aligning with resource-constrained edge computing environments.

3.4. Agent Network Protocol (ANP)

Agent Network Protocol (ANP) is an open-source agent protocol developed to enable interoperability among agents across heterogeneous domains. ANP presents a decentralized vision for agent communication and adopts a peer-to-peer (P2P) architecture designed from the ground up for decentralization. The ANP architecture consists of three layers: the Identity and Encrypted Communication layer, the Meta-Protocol layer, and the Application Protocol layer [33].

The Identity and Encrypted Communication layer leverages W3C Decentralized Identifiers (DIDs) [41] to establish identity authentication and enable end-to-end encrypted communication across different platforms/domains. A DID provides a cryptographically verifiable, self-sovereign identity that is not controlled by any central authority [42].

Instead of being restricted to a fixed set of predefined schemes for inter-agent information exchanges, the Meta-Protocol layer in ANP allows agents to describe their requirements and constraints in natural language, which are then processed by LLMs for protocol negotiation, to enable agents to dynamically adjust communication parameters, define data formats, or even collaboratively generate the necessary protocol on the fly. On the other hand, to reduce communication costs, agents should avoid protocol negotiation whenever possible. Therefore, ANP designs a series of semantic Web standard-based schemes on the Application Protocol layer to make inter-agent communication more efficient and cost-effective.

A key function of the Application Protocol layer is agent description and discovery. This layer uses JSON-LD [43] to achieve semantic interoperability. By embedding linked data in a standard JSON format, JSON-LD allows agent descriptions to be both easily parseable by machines and semantically rich. ANP specifies a flexible, dual-mechanism approach for agent discovery. The active discovery method enables an agent to discover available agents within a known domain without a central registry by polling a standard URI: https://domain/.well-known/agent-descriptions. The passive discovery method creates a central or federated search service agent to which the available agents can submit the URLs of their agent descriptions.

3.5. Language Model Operating System (LMOS)

The Language Model Operating System (LMOS) is an open-source specification developed by the Eclipse Foundation for building and running enterprise-ready multi-agent systems [34]. Its core innovation is not the invention of a new agent protocol from scratch, but the strategic adaptation of the W3C Web of Things (WoT) standard [44] as the foundational architecture for inter-agent communication, thereby providing LMOS with mature, standardized solutions for agent description, discovery, and communication.

In LMOS, AI agents and tools are described using the WoT Thing Description (TD), a machine-readable, JSON-LD formatted document that semantically describes a "Thing" and its network-facing interfaces. The key information about an agent in TD includes

Properties (data the agent exposes), Actions (functions the agent can perform), and Events (notifications the agent can emit).

The WoT-based agent discovery in LMOS supports three primary discovery methods. The centralized method allows agents to publish their TDs to a registry, from where any agent in the network can perform semantic queries to discover other agents. The local discovery method in LMOS enables agents to employ mDNS (Multicast DNS) [45] or DNS-SD (DNS Service Discovery) [46] to advertise their presence and discover peers on local area networks. LMOS also supports peer-to-peer metadata propagation, thereby eliminating reliance on a single registry and enabling decentralized agent discovery.

A key attribute of LMOS is its protocol-agnostic nature. The TD for agent description separates the abstract inter-agent interaction from its concrete protocol implementation. This is achieved through forms specified in the TD, which provide protocol bindings that specify how agent communications use various transport protocols, such as HTTP, WebSockets [47], Message Queuing Telemetry Transport (MQTT) [48], and Constrained Application Protocol (CoAP) [49]. This allows a single LMOS agent to expose its capabilities simultaneously via HTTP (for a cloud orchestrator) and MQTT (for a local edge message bus), thereby ensuring maximal interoperability and flexibility. The LMOS also provides high-level orchestration functions, including intelligent task management and routing, for resource allocation and load balancing at the MAS level, which provide guidelines for agent communications.

3.6. Agent Connect Protocol (AConP) from the AGNTCY Project

The AGNTCY project, with the explicit goal of building a modular, composable infrastructure stack for an open, decentralized Internet of Agents, was announced by Cisco Outshift in March 2025 and developed in collaboration with LangChain and Galileo. In July 2025, Cisco formally donated the AGNTCY project to the Linux Foundation to ensure neutral governance and encourage industry-wide adoption. The AGNTCY project provides a suite of components, including Open Agent Schema Framework (OASF), Agent Directory Service (ADS), Secure Low-latency Interactive Messaging (SLIM), and Agent Connect Protocol (AConP), to address different aspects of the agent communication problem [50].

OASF is an extensible data model based on the Open Container Initiative (OCI) specification [51] that provides a standardized ontology for describing an agent's attributes, capabilities, and metadata. ADS is a distributed directory system designed to store and manage agent descriptions using OASF-defined schemas and to employ a distributed hash table (DHT) for scalable agent discovery. AConP is a REST-based API specified in OpenAPI [52] that defines a standard interface for agent invocation and configuration, with primary operations focused on the lifecycle management of task executions and conversational context. SLIM extends gRPC's standard request/reply and streaming patterns with native publish-subscribe (pub/sub) capabilities. SLIM also enables efficient transmission of binary payloads, thereby better supporting lightweight communications in resource-constrained environments.

The AGNTCY project presents a decentralized system for managing and verifying the identities of agents, tools, and multi-agent systems by leveraging W3C DIDs and Verifiable Credentials (VCs) [53]. In AGNTCY, a trusted issuer (e.g., an organization such as Google) cryptographically signs an "Agent Badge", which is a W3C VC that attests to an agent's claims. This Agent Badge contains the agent's DID, its OASF schema definition, and its provable capabilities and provenance. When two agents need to start a communication session, one can present its VC, and the other can cryptographically verify the issuer's signature and trust the claims without contacting a central authority.

3.7. Merger of A2A and ACP

The parallel development of multiple protocols, each targeting distinct agent communication environments, introduces a new interoperability challenge at the protocol level in the agentic AI ecosystem. For instance, an "edge agent" running ACP could not readily offload a computationally intensive task to a "cloud agent" using A2A, necessitating complex translation layers or "bridge agents". On the other hand, all protocols share a fundamental goal: enabling agents to communicate across frameworks, organizations, and technology stacks [54].

The merger of A2A and ACP under the neutral governance of the Linux Foundation (LF AI&Data) to form a new A2A protocol, as announced in August 2025, is a recent community effort to address this challenge [55]. The merged A2A protocol adopts a hybrid architecture that integrates ACP's edge-native resilience with A2A's cloud-oriented design and also incorporates some features from other protocol developments (e.g., the AGNTCY project). In this section, we highlight the key enhancements introduced in the merged A2A protocol across three areas: agent description, agent discovery, and information transport.

The merged protocol embraces OASF, originally contributed by the AGNTCY project, and integrates the ACP's metadata schema into its Agent Card mechanism, which is inherited from A2A. The Agent Card is now a JSON-LD document with a schema that introduces critical fields for operational context, including agent resource constraints in addition to agent identity and capability information. This integration addresses a major deficiency in the legacy A2A and significantly enhances the protocol's capability to support interoperability across diverse agents and heterogeneous underlying infrastructures.

The new A2A protocol implements a dual-mode agent discovery system that reconciles the online requirements of the cloud with the offline realities of edge environments. For active, network-connected agents, the protocol performs online discovery using the enhanced Agent Cards published in a registry or at a well-known URI. The protocol also supports an offline agent discovery model. In this mode, the Agent Card metadata is serialized and embedded directly in the agent's distribution artifact, enabling client agents to discover available remote agents without waking their radios or processors.

The merged A2A protocol adopts a protocol negotiation strategy that supports multiple transport schemes, optimizing for the specific network environment [56]. The transport mechanisms supported by the new A2A include JSON-RPC 2.0, inherited from the legacy A2A, which is suitable for cloud-based enterprise environments, the REST-native messaging adopted in legacy ACP, which is more appropriate for edge/IoT applications, and the gRPC-based SLIM scheme originally developed in AGNTCY for low-latency and high-throughput message transport. The shift from a "one-size-fits-all" monolithic transport approach to a "meta-protocol" mechanism that adaptively utilizes an appropriate scheme based on communication requirements and deployment environments may greatly improve the protocol's performance in the highly dynamic future Internet.

4. Key Technologies for Agent Communications in Future Internet

In this section, we review the key technologies employed by representative inter-agent communication protocols to assess their readiness to the challenges of building an Agentic Web in the future Internet. We first discuss the main criteria for assessing agent communication technologies in terms of their capabilities to address the challenges of heterogeneity, scalability, dynamicity, efficiency, and security, and then present an analysis of the technologies for agent description, publication and discovery, identification and authentication, information representation, and information exchange.

4.1. Criteria for Assessing Agent Communication Technologies

The key criteria for assessing the capabilities of agent communication technologies to meet the main challenges introduced by the Agentic Web in the future Internet are as follows.

Heterogeneity: The main criteria for assessing the effectiveness of an agent communication technology in addressing the heterogeneity challenge include interoperability with diverse agents and compatibility with various infrastructures. The primary criterion for the interoperability aspect is the depth of semantic expression, which ensures that the meaning and intent of communication are preserved across domain boundaries. The key factor in assessing compatibility is the degree of protocol agnosticism, which often involves abstracting interfaces and decoupling functions from specific implementations.

Scalability: To evaluate the scalability of an agent communication technology, the key criteria focus on how well a technology maintains its performance, such as throughput and delay, while the MAS expands (i.e., both the number of participating agents and the number of inter-agent connections increase). Such criteria include both procedural and structural aspects—the former focuses on how process complexity increases with system scale and thus potentially degrades system performance, while the latter focuses on whether the technology's coordination mechanisms form a potential bottleneck or a single point of failure as the system expands.

Dynamicity: Assessing an agent communication technology's readiness for the dynamicity challenge focuses on how adaptable it is to changes in system states (e.g., resource availability) and to shifts in demand or workload. Key evaluation criteria include the technology's resilience to environmental volatility, such as intermittent connectivity or node mobility, and its capability for graceful degradation and elastic reconfiguration, ensuring that agents can maintain core functionality in response to real-time hardware fluctuations or partial system failures.

Efficiency: Evaluation of the efficiency of an agent communication technology centers on computational and communication parsimony to maximize utility while minimizing resource consumption. The key assessment criteria are the overheads incurred by the technology in realizing its functionality. This includes both computational overheads for protocol processing, such as the computing power and storage space consumed, and communication overheads, such as bandwidth consumption. The computational and communication overheads together determine the technology's energy efficiency, a critical criterion for agent communication in resource-constrained edge environments.

Security: Assessment of the security of an agent communication technology primarily examines its effectiveness in protecting the confidentiality of information shared among agents, maintaining the integrity of inter-agent interactions, and ensuring the MAS's availability. Specific criteria include the technology's ability to support trust management in collaborative multi-agent environments and to mitigate specialized security threats to MASs in the Internet, such as prompt injection and Sybil attacks.

The aforementioned evaluation criteria are employed in our qualitative assessment of key agent communication technologies in the rest of this section. In each of the following subsections, we first categorize the typical technologies for realizing the key functionalities of agent communications, namely, agent description, agent publication and discovery, agent identification and authentication, information representation, and information exchange; and then analyze the effectiveness of each technology category in facing the heterogeneity, scalability, dynamicity, efficiency, and security challenges. Then, we give a summary of our analysis in Table 1 at the end of this section.

Table 1. Key technologies for agent communication and their capabilities of addressing challenges in the future Internet.

Technology Categories	Capabilities of Addressing Internet Challenges				
	Heterogeneity	**Scalability**	**Dynamicity**	**Efficiency**	**Security**
Agent Description					
Syntactic	Low	Medium	Medium	High	Low
Semantic	High	Medium	High	Low	Medium
Agent Publication & Discovery					
Centralized	High	Low	Low	Low	Medium
Decentralized	Medium	High	High	Medium	Low
Agent Identification & Authentication					
CA-based	High	Low	Low	Low	High
Peer-to-peer	Medium	High	Medium	Medium	Medium
Information Representation					
Structured	Medium	High	Medium	High	High
Unstructured	High	Medium	High	Low	Low
Information Exchange					
Synchronous	Medium	Medium	Low	Medium	Medium
Asynchronous	High	Medium	High	High	Low

4.2. Agent Description

Representative approaches employed by common agent communication protocols for agent description can be categorized as syntactic, semantic, and API schema-based.

4.2.1. Syntactic-Based Methods

A syntactic description focuses on the form, structure, and grammar of metadata that describe AI agents' attributes and capabilities, providing a baseline agent description for interoperability. Syntactic description methods often use JSON files to construct Agent Cards or manifests. Another common approach to syntactic descriptions uses the OpenAPI-based schema [52] to describe the interfaces of AI agents. Both schemes have been widely adopted across virtually all agent communication protocols.

Heterogeneity: The lack of semantic expression in both JSON files and OpenAPI schema limits their ability to handle ambiguity in heterogeneous agents' descriptions. Also, JSON files and API schemas often lack a standard for describing non-functional constraints (e.g., the host device's computing capacity), making it difficult to perform resource-aware agent discovery and selection across the heterogeneous edge–network–cloud continuum.

Scalability: Syntactic agent descriptions are typically lightweight and easy to index; therefore, they can better support high-volume queries than complex ontological descriptions. On the other hand, the constraint on syntactic information limits its ability to support scaling communications across diverse agents.

Dynamicity: OpenAPI is designed to describe static interfaces and does not inherently support dynamic updates. While the JSON file itself is static, its fields can be updated to adapt to changes in agent capabilities and deployment environments. However, real-time state changes (like battery drain) are better handled by a separate status protocol than by constantly updating a static manifest file.

Efficiency: JSON parsers are ubiquitous and highly optimized, minimizing overhead for both data processing and transmission. Code generators can compile OpenAPI specifications into efficient, native client libraries, eliminating the need for runtime schema introspection. Therefore, syntactic descriptions can be processed efficiently and also facilitate efficient agent discovery and selection.

Security: Syntactic methods are vulnerable to command injection and logic-level exploits because they do not carry the semantic intent required for deep validation. They lack built-in mechanisms for fine-grained authorization, often leading to broad, over-privileged access tokens that increase the risk of lateral movement if an agent is compromised.

4.2.2. Semantic Descriptions

A semantic description focuses on describing the meaning, intent, and context of the agent's capabilities using shared vocabularies and ontologies. JSON-LD with the @context field is a common format for presenting semantic agent descriptions, which has been employed in ACP, ANP, and LMOS. The OASF, which leverages a custom JSON-based record format inspired by the Open Cybersecurity Schema Framework (OCSF) [57], also provides semantic agent descriptions.

Heterogeneity: Semantic formats such as JSON-LD are explicitly designed to address the heterogeneity problem. By linking data to shared ontologies, they ensure that an agent from one vendor can interpret the capabilities of an agent from another vendor unambiguously.

Scalability: While the semantic descriptions are text and scale well, the reasoning required to process them does not. As the number of agents grows, maintaining and synchronizing shared ontologies across the large-scale Internet becomes a significant management bottleneck.

Dynamicity: JSON-LD allows for flexible, decentralized extensibility. An agent can dynamically add new capability terms to its description without breaking existing parsers, which is vital for the evolution of multi-agent systems in the dynamic Internet environment.

Efficiency: Parsing and validating formal semantics is computationally expensive. JSON-LD, in particular, introduces high processing overhead for resolving linked data contexts, which can be prohibitive in resource-constrained edge computing environments.

Security: Semantic descriptions enable robust context-aware authorization, allowing systems to verify if an agent's requested action aligns with its defined purpose. On the other hand, additional processing for XML or complex JSON-LD graphs might create a larger attack surface, e.g., for Denial-of-Service (DoS) and DNS poisoning and hijacking.

4.3. Agent Publication and Discovery

Technologies for agent publication and discovery make agents' descriptions available to others. Typical publication and discovery mechanisms in common agent protocols can be classified as centralized or decentralized based on their control structures.

4.3.1. Centralized Methods

Centralized publication and discovery approaches require agents to actively register their descriptions with a central server or store them at a standardized web location, from which other agents can search and find agents that meet their requirements. The centralized approach has been leveraged in A2A, ACP, and LMOS.

Heterogeneity: A central registry can enforce a common description schema, such as an Agent Card or OASF, enabling heterogeneous agents to describe themselves and discover others in a standardized, interoperable way, thereby greatly enhancing the system's ability to address heterogeneity.

Scalability: Centralized agent publication and discovery methods may suffer from scalability issues due to performance bottlenecks and a single point of failure, especially

in large-scale Internet deployments, where a vast number of highly distributed agents are deployed.

Dynamicity: Centralized methods require agents to explicitly register and update their descriptions, which becomes infeasible in highly dynamic networks where agents on mobile devices constantly change locations or lose connectivity. Furthermore, discovery fails entirely if the central server or location becomes unreachable, for example, due to a network or power outage.

Efficiency: With a centralized approach, every discovery action requires a round-trip communication session to a (potentially distant) central location, resulting in higher discovery latency and additional network overhead that reduces the efficiency of agent communication.

Security: Centralization enables authoritative vetting and auditing of agents, reducing the risk of malicious agents entering the ecosystem. However, a central registry is a high-value target for DoS attacks or compromise, and a breached registry could allow an attacker to redirect all discovery requests to malicious agents.

4.3.2. Decentralized Methods

Decentralized publication and discovery methods disseminate agent descriptions without a central server or web location, typically via peer-to-peer communication or local broadcast/multicast. This strategy has been adopted in ANP and AGNTCY, and the offline discovery in ACP and merged A2A is also considered a decentralized method.

Heterogeneity: Decentralized publication/discovery mechanisms focus on disseminating agent descriptions without imposing a common schema; therefore, they typically require semantic descriptions, such as JSON-LD, to ensure interoperability across heterogeneous agents.

Scalability: Decentralized approaches distribute the discovery process across agents, eliminating performance bottlenecks and single points of failure, and thereby greatly enhancing the scalability of agent communication systems.

Dynamicity: The peer-to-peer messaging or local multicast/broadcast used in decentralized agent discovery allows individual agents to update their descriptions as needed, making the system more adaptive to system dynamics. Offline discovery is especially well-suited for intermittently connected devices common in dynamic networks, such as IoTs.

Efficiency: On the one hand, decentralized discovery in local networks has very low latency, and offline discovery even generates zero network traffic. On the other hand, naive peer-to-peer messaging or network-wide broadcasts could incur excessive communication overhead in resource-constrained edge computing environments.

Security: Decentralized discovery is resilient against single points of failure but highly vulnerable to Sybil attacks, where an attacker creates multiple fake identities to manipulate discovery outcomes [58]. Without a central authority, verifying the legitimacy of discovered agents becomes difficult, increasing the risk of interacting with impersonated or spoofed entities.

4.4. Agent Identification and Authentication

Representative technologies for the assignment and verification of agents' identities to establish trust in agent communication systems typically employ either central authority (CA)-based methods or cryptography-based peer-to-peer methods.

4.4.1. CA-Based Authentication

Identification/authentication methods in this category rely on a central Identity Provider (IdP) to issue and validate credentials for establishing trust among autonomous

agents. A common implementation of this strategy is the OAuth 2.0-based scheme [59] used in A2A and ACP.

Heterogeneity: OAuth 2.0 is a mature, ubiquitous standard supported by all cloud platforms and most enterprise information systems. Therefore, it facilitates interoperability among autonomous agents to face the heterogeneity challenge.

Scalability: This authentication model is architecturally bound to a central IdP for token issuance and validation, which becomes a performance bottleneck and a single point of failure, potentially degrading the scalability of agent communications.

Dynamicity: CA-based methods require each agent to contact a central IdP to obtain and refresh its time-bound access token; therefore, authentication may fail if the IdP becomes unreachable, which can affect performance in highly dynamic scenarios such as IoT with intermittent connections.

Efficiency: The regular communication between each agent and the IdP required for token acquisition and validation introduces not only latency in agent authentication but also additional bandwidth consumption, which may reduce efficiency for agent communications.

Security: Mature mechanisms such as OAuth 2.0 provide standardized procedures for consent and token revocation, which are essential for managing trust. However, authentication tokens are vulnerable to interception and replay attacks; a stolen token enables an attacker to impersonate the agent for the token's lifetime.

4.4.2. Peer-to-Peer Authentication

Decentralized peer-to-peer identification and authentication allow individual agents to control their own identities and verify others' identities without relying on any central authority. Both ANP and AGNTCY (AConP) embrace this strategy in their DID/VC-based agent identity/authentication schemes.

Heterogeneity: DID is a W3C standard widely adopted in edge and cloud computing, therefore facilitating interoperability across heterogeneous agents. However, the standard requires complex privacy key management and cryptographic operations, which may be infeasible in heterogeneous edge–network–cloud environments comprising a large number of resource-constrained devices.

Scalability: Peer-to-peer authentication methods improve scalability in agent communications by allowing individual agents to create their own identifiers and verify credentials, thereby avoiding potential performance bottlenecks and single points of failure.

Dynamicity: By allowing self-contained agent identities that can be verified by agents' digital signatures and resolved by peers through their public DID documents, peer-to-peer authentication methods are well-suited to the dynamic edge environments of the future Internet.

Efficiency: Once the public DID document is resolved, decentralized authentication is a local cryptographic operation that is more efficient than the network-bound OAuth validation required by CA-based methods. On the other hand, privacy key management and cryptographic processes may be computationally overwhelming for resource-constrained devices, especially at the network edge.

Security: Peer-to-peer authentication eliminates the risk of a centralized IdP and enables a "trustless" ecosystem where trust is established via cryptographically verifiable credentials. On the other hand, system security relies on individual agents' ability to protect their private keys; edge devices without robust security mechanisms are particularly vulnerable to credential theft.

4.5. Information Representation

Information representations in agent communications define the formats for presenting the information exchanged between AI agents. Technologies for achieving this objective in representative inter-agent communication protocols leverage either structured data formats or unstructured natural language.

4.5.1. Structured Data Formats

The structured data formats used in agent communication protocols for information representation include JSON and its extension JSON-LD. JSON is a lightweight, text-based data format with no built-in semantics, whereas JSON-LD is designed to represent linked data across systems using URIs and to enrich JSON data with semantic meaning. Structured data can be transported between agents via either JSON-RPC (as in the A2A and ANP protocols) or RESTful JSON messaging (as in ACP, LMOS, and AConP).

Heterogeneity: Simple JSON is a syntactic standard that lacks the formal semantics needed to resolve ambiguity across agents' specific schemas, leading to poor interoperability among heterogeneous AI agents. JSON-LD carries both data and its formal, logical meaning, ensuring unambiguous interpretation and thus significantly enhancing its ability to address the heterogeneity challenge in agent communication. While the function-specific interface of JSON-RPC may hinder interoperability across heterogeneous agents, RESTful JSON uses the universal HTTP messages (e.g., GET, POST, PUT, DELETE) and standard URIs to provide uniformity, allowing various agents hosted on heterogeneous edge–cloud devices to interact.

Scalability: JSON and JSON-LD have complementary effects on the scalability of agent communications. JSON is well-suited for agent messaging at scale due to its low serialization overhead and fast parsing, but at the cost of poor interoperability. Although JSON-LD enables semantic interoperability, facilitating scalable discovery and capability matching across diverse agents, it also increases message size and computational overhead for context processing and reasoning. Therefore, using JSON for runtime message exchange and JSON-LD for control-plane functions, such as agent description and discovery, is an appropriate strategy to address the scalability challenge.

Dynamicity: The self-describing nature of JSON/JSON-LD enables dynamic, adaptive data queries and updates between agents and is therefore well-suited to highly dynamic edge computing systems, where nodes frequently join, leave, or evolve. Furthermore, LLMs are natively optimized for generating and parsing JSON, enabling an LLM-driven agent to dynamically create new message structures to adapt to emerging situations. The stateless RESTful JSON allows an agent to resume interrupted communications without restoring session state, thereby facilitating system resilience. On the other hand, RESTful JSON lacks native support for real-time "push" updates, requiring inefficient polling to handle dynamic events.

Efficiency: JSON provides low serialization overhead, fast parsing, and broad tooling support, making it efficient for information transport between AI agents. JSON-LD introduces additional processing and parsing overhead for interpreting its linked-data graph structure, making it unsuitable for resource-constrained scenarios such as IoTs. Therefore, choosing data formats that best fit the type of information exchanged and the deployment environment is critical to maintaining efficiency in agent communication.

Security: Structured data formats facilitate security protection for agent communication by allowing input data to an agent to be validated against deterministic schemas, thus making it easier to sanitize against injection attacks.

4.5.2. Unstructured Natural Languages

Since virtually all modern AI agents are built on LLMs with strong natural language processing capabilities, human languages provide a natural means of representing information in inter-agent communication to support multi-agent collaboration [60].

Heterogeneity: Natural language functions as a "universal solvent" for interoperability across heterogeneous AI agents. Unlike rigid protocols that require a prior schema integration, natural language allows agents to use LLMs to infer intent across disparate terminologies. This capability enables highly heterogeneous agents to collaborate spontaneously, effectively addressing the heterogeneity challenge.

Scalability: While natural language supports the open-ended collaboration conceptually, its scalability in practice is constrained by LLMs' limited context windows. As the number of agents increases, the conversation history grows super-linearly, forcing agents to compress or "forget" information, leading to a high "coordination tax" and information fragmentation [61]. Consequently, natural language-based information representations struggle to scale in highly distributed networks.

Dynamicity: Natural language offers exceptional adaptability in volatile network environments. It allows agents to dynamically discover peers and negotiate capabilities using high-level intents rather than static endpoints, making agent communication resilient to changes in both agents' capabilities and infrastructure resources. Furthermore, agents can utilize natural language (e.g., as in the Agora framework) to negotiate new structured data formats, thereby further enhancing the system's adaptability.

Efficiency: Natural language is inherently verbose, and processing it generates greater computational overhead and consumes more network bandwidth than structured formats. More critically, processing natural language using LLMs replaces deterministic, microsecond-scale parsing with probabilistic, millisecond-to-second-scale inference, creating an energy and latency bottleneck that makes it unsuitable for real-time control loops or resource-constrained edge devices.

Security: Natural language is the primary vector for prompt injection and goal hijacking, as agents cannot reliably distinguish between system instructions and malicious user-injected content. Traditional defenses fail because natural language operates at the semantic layer rather than the network layer, making it impossible to validate using standard signature-based methods.

4.6. Information Exchange

Information exchange technologies control the patterns for exchanging information between AI agents. The main approaches to information exchange in common agent communication protocols can be categorized as synchronous and asynchronous.

4.6.1. Synchronous Mode

Synchronous information exchange approaches follow a request/response pattern in which an agent sends a request message to a remote agent and waits for a response. Such a request/response process may be implemented using a variety of schemes, including HTTP REST calls, JSON-RPC, and gRPC.

Heterogeneity: On the one hand, request-response messaging schemes are well-defined standards that are easy to implement in AI agents, thereby supporting interoperability among agents. On the other hand, synchronous information exchange is essentially a tightly coupled interaction, which poses challenges for communication among highly diverse AI agents.

Scalability: Although the stateless nature of the synchronous exchange pattern facilitates system scalability, it focuses on point-to-point connections between pairs of agents

and may thus constrain scalability for communications across the massive number of AI agents deployed in the large-scale Internet.

Dynamicity: Synchronous information exchange approaches require a stable connection throughout the communication session, which may not be guaranteed in highly dynamic environments such as wireless mobile networks and edge computing systems. The tightly coupled interactions for synchronous information exchange are also ill-suited to agents hosted on mobile devices that may regularly move (and thus change IP addresses) or go offline.

Efficiency: The efficiency of synchronous information exchange approaches varies with their specific implementation schemes. Among the typical schemes, gRPC is the most efficient and suitable for low-latency data transmission within a data center; HTTP REST calls have the highest overhead due to verbose (JSON or XML) payloads and heavy HTTP headers; and JSON-RPC lies between gRPC and HTTP REST calls in efficiency.

Security: Synchronous information exchange allows real-time monitoring and termination of the communication sessions, thus facilitating security protection. On the other hand, synchronous exchanges are susceptible to session-hijacking attacks, and their real-time nature also makes them a target for resource-exhaustion DoS attacks.

4.6.2. Asynchronous Mode

Asynchronous information exchange approaches avoid tightly coupled interactions between agents, enabling more flexible communication patterns. Asynchronous information exchange can be implemented using various schemes, including publish/subscribe, asynchronous streaming, and asynchronous queueing. The publish/subscribe scheme allows an agent (publisher) to send a message about a "topic" to a broker, which then delivers it to all agents that have subscribed to that topic. With asynchronous streaming, an agent (sender) opens a connection and pushes a continuous stream of messages to another agent (receiver). Using asynchronous queueing, a sender agent may push a message to a queue, from where a receiver agent can retrieve it later as needed.

Heterogeneity: The loose-coupling interactions enabled by asynchronous approaches allow information exchange between agents without being constrained by their implementations or hosting platforms, greatly facilitating communication across heterogeneous AI agents.

Scalability: Asynchronous approaches overcome the constraints of point-to-point sessions in the synchronous request-response pattern and are thus better suited to many-to-many information exchanges, thereby enhancing the scalability of agent communications. On the other hand, both the publish/subscribe broker and the asynchronous queue must be carefully designed to avoid creating a performance bottleneck that could degrade system scalability.

Dynamicity: Asynchronous information exchange approaches implemented with a publish/subscribe broker or an asynchronous queue are particularly well suited to highly dynamic communication environments with intermittent connectivity. With such approaches, a mobile edge agent can use a temporary network connection to publish data to a broker (or push it into a queue), which then delivers it to other agents (subscribers) when their connections are available. Agents can also dynamically update their subscriptions to various topics.

Efficiency: Asynchronous approaches are typically considered more efficient for information exchange than synchronous ones in most inter-agent communication scenarios. For example, MQTT, a common scheme for publish/subscribe operations, enables lightweight message delivery optimized for resource-constrained network environments such as IoTs and edge computing systems.

Security: Asynchronous messaging is highly vulnerable to replay attacks, in which a valid message is captured and later resent to repeat an action, such as a financial transaction. Although the message broker can act as a checkpoint for authentication, it also becomes a central target for eavesdropping or message tampering.

5. Comparative Analysis of Agent Communications Protocols

Each agent communication protocol is a bundle of technologies that are integrated within a framework, following certain design principles, to achieve the protocol's objectives. Based on the technology assessment presented in Section 4, we provide a comparative analysis of representative inter-agent communication protocols in this section to evaluate their effectiveness in supporting multi-agent collaboration in the Agentic Web. We first review the key attributes of these protocols and then use the example MASs described in Section 2.2—the cooperative Smart Transport and Health (STH) system and the open Collaborative Research Community (CRC) system—as use cases to conduct a scenario-driven analysis for each protocol.

5.1. Key Attributes of Agent Communication Protocols

5.1.1. A2A (Agent2Agent Protocol): A Cloud-Native Baseline

A2A is a representative agent communication protocol designed to support agentic AI applications in enterprise environments. Its technology bundle, leveraging HTTP, JSON-RPC, OAuth, and a centralized registry, is optimized for cloud-native data centers or enterprise networks, not the edge–network–cloud continuum across the Internet, and thus is not well prepared to address all the challenges introduced by the future Internet. In the Internet environment, the protocol may fail to scale due to its centralized registry bottleneck, suffer insufficient adaptability due to its online-only discovery and synchronous-first interaction schemes, and degrade communication efficiency due to HTTP/JSON-RPC overheads. While its opaque execution enables interoperability across diverse AI agents, the lack of infrastructure-related information in its Agent Card limits the protocol's ability to handle device-level heterogeneity.

5.1.2. ACP (Agent Communication Protocol): An Edge-Native Contender

ACP is a protocol explicitly designed with local/edge autonomy as its focus, as reflected in its technology choices. It prioritizes efficiency with its lightweight REST-native architecture and supports dynamicity through its async-first design with an offline agent discovery model. Its distributed architecture facilitates scalability; however, its "local-first" focus introduces scalability limitations (e.g., the absence of a scalable mechanism for agent discovery). A primary weakness of this protocol is its limited capacity to accommodate heterogeneity; it lacks built-in support for semantic agent descriptions, which risks brittle, ad hoc agent interactions and thus limits its performance in the future Internet, where a wide variety of AI agents are expected to be deployed.

5.1.3. ANP (Agent Network Protocol): A Decentralized Vision

ANP is an agent communication protocol designed following a decentralized vision. The key technologies leveraged by this protocol, for example, W3C DIDs for agent authentication and a P2P structure for agent discovery, are, in principle, well-suited to addressing scalability and dynamicity challenges. Its use of JSON-LD provides a strong solution for heterogeneity. However, the strengths of this protocol's forward-looking, decentralized architectural vision are offset by some designs ill-suited to the realistic edge–network–cloud continuum expected in the future Internet. The protocol may incur significant efficiency losses and moderate scalability limitations in resource-constrained edge computing envi-

ronments, primarily due to its high computational overhead, for example, parsing JSON-LD and performing cryptographic operations for DIDs.

5.1.4. LMOS: A Hybrid Orchestration Model

The LMOS protocol aims to achieve a balance for a converged edge–network–cloud environment in the future Internet, primarily through an architectural framework that embraces pragmatic hybridity. It addresses scalability and dynamicity challenges by supporting both centralized and P2P mechanisms for agent discovery, and employing both enterprise OAuth and decentralized DIDs for agent identification/authentication. The protocol addresses heterogeneity through its transport abstraction layer, which bridges diverse network environments, combined with JSON-LD for semantic richness. The main weakness of this protocol, like ANP, lies in the potential efficiency costs introduced by some of its key functionalities, for example, parsing JSON-LD, on resource-limited edge/IoT devices.

5.1.5. AGNTCY: A Full-Stack Approach

Rather than a single protocol, AGNTCY offers a full-stack solution for agent communications, comprising four interrelated core components: OASF, ADS, SLIM, and AConP. The OASF allows agents to describe their operational context alongside functional skills, thereby enabling resource-awareness in agent description and discovery, which is required to address the heterogeneity challenge. The ADS implements a decentralized agent-discovery mechanism that is highly resilient to the large-scale edge–network–cloud continuum. The SLIM messaging scheme, built on gRPC and HTTP/2, offers a significant efficiency advantage over text-based JSON-RPC/RESTful JSON calls. On the other hand, the AGNTCY architecture remains essentially cloud-centric and has not fully addressed opportunistic communication, delay-tolerance, or adaptive quality-of-service strategies required in resource-constrained and dynamic edge computing systems.

5.1.6. Merged A2A Protocol: A Unified Standard

By synthesizing A2A's cloud-centric design with ACP's edge-native solution, the merged A2A protocol addresses the critical fragmentation between cloud and edge environments. By integrating A2A's opaque task-delegation model with ACP's embedded metadata and device-native capabilities, the protocol substantially improves interoperability through a dual-stack solution, which preserves JSON-RPC compatibility while introducing REST-aligned patterns suitable for constrained edge devices. The protocol enhances scalability by supporting both centralized and decentralized discovery with ACP's offline mechanism incorporated. The merged protocol adopts ACP's async-first design philosophy, making it resilient to intermittent connectivity in highly dynamic mobile edge environments. A hybrid authentication model supporting both OAuth and DIDs, ensuring trust management across both connected and disconnected operational contexts. Efficiency is also improved over the legacy A2A by adopting ACP's multi-part message structure, which allows handling of binary data without the overhead of pure JSON-RPC. On the other hand, the highly diverse requirements of agentic AI applications necessitate deploying various multi-agent systems across heterogeneous infrastructures, making the "one-fit-all" strategy implemented through a single protocol for all agent communication scenarios less feasible in the future Agentic Web.

5.2. Scenario-Driven Evaluation of Agent Communication Protocols

In this section, we use the Smart Transport and Healthcare (STH) and the Collaborative Research Community (CRC) systems presented in Section 2.2 as two use cases to evaluate the agent communication protocols for Agentic Web. For each protocol, we not only assess

its performance in supporting individual STH and CRC systems but also examine its multi-tenancy ability to operate both systems in parallel in the Agentic Web.

The STH system is characterized by high-stakes, real-time coordination that requires low-latency interactions among mobile edge agents (ambulances), stationary edge agents (e.g., RSUs), and centralized cloud agents (hospital data centers). The primary communication modes include synchronous request-response for resource allocation and real-time streaming for navigation and traffic control. Conversely, the CRC scenario emphasizes massive scalability and extreme heterogeneity, involving hundreds of agents ranging from resource-constrained sensors to high-performance computing clusters. Interaction in the CRC is predominantly asynchronous, leveraging publish-subscribe or message-queueing patterns to accommodate intermittent connectivity and varying processing capacities among participants.

5.2.1. Agent2Agent (A2A) Protocol

The A2A protocol, as a cloud-native baseline, offers a framework optimized for enterprise environments. When applied to the STH scenario, A2A effectively supports the high-level orchestration between hospital data centers and the ambulance's client agent. The Agent Card mechanism allows healthcare facilities to publish precise treatment capabilities, enabling ambulances to discover and select the optimal facility via a registry-based discovery method. However, reliance on JSON-RPC 2.0 over HTTP/HTTPS in A2A may incur unacceptable overhead in the STH scenario. As illustrated by quantitative benchmarks, a typical A2A interaction over standard RESTful patterns may have a median delay of 250 ms [62], which does not meet the real-time control requirement for emergency response. The text-based JSON serialization in A2A is computationally expensive, for example, resulting in a 30–50% larger payload than that of binary formats and increasing bandwidth consumption and processing costs.

In the CRC scenario, A2A's opaque execution model is highly beneficial for research agents hosted by diverse organizations. It allows lead agents to delegate complex analysis tasks to specialized agents without requiring the exchange of proprietary data models or internal logic. Nevertheless, A2A fails to support the massive scale of sensor agents in the CRC. The requirement that each agent in the CRC system maintain an active Agent Card in a central registry would trigger a signaling storm that overwhelms the network's control plane. The protocol's token cost is also a concern in the CRC scenario, as JSON's verbosity leads to significant overhead. For example, benchmark results have shown that JSON consumes 30–60% more tokens than a more compact format such as TOON [63].

When STH and CRC operate in parallel under the A2A, the protocol faces severe resource contention due to its lack of a native prioritization or resource-aware scheduling mechanism. Therefore, a surge in sensing traffic in the CRC system could delay the delivery of real-time emergency medical requests in the STH system.

5.2.2. Agent Communication Protocol (ACP)

ACP's edge-native design provides an alternative for the STH scenario. The protocol's lightweight REST-native approach allows RSUs to handle traffic signal requests with minimal computational overhead. The offline agent discovery supported by ACP ensures that ambulances and RSUs can maintain local-first autonomy, coordinating emergency movements even if the Internet infrastructure is compromised. However, its limited semantic expression makes the protocol struggle with the high-level reasoning required for complex healthcare logistics. For example, coordinating an ambulance route that accounts for both traffic signals and the availability of specialized surgeons across multiple hospitals requires a deep semantic understanding of agent intents. Furthermore, ACP's

RESTful architectural style poses challenges for the STH system's real-time requirements. Its stateless nature often requires a new TCP/TLS handshake for each request, adding 50-100 ms of extra latency per message as indicated by quantitative analysis [64].

ACP's simple, RESTful, edge-native design makes it well-suited to the CRC scenario. The asynchronous nature of CRC aligns with ACP's design, allowing long-running analytic tasks to be monitored via standard HTTP polling or webhooks. However, the lack of semantic expression of ACP may lead to vocabulary fragmentation when different agent groups use different terms for the same data point, thus constraining the protocol's performance in the CRC scenario.

In parallel operation, ACP's asynchronous nature prevents low-priority CRC tasks from blocking high-priority STH requests. However, the lack of a standardized cross-domain discovery mechanism in ACP means that the STH and CRC systems remain entirely isolated, preventing an STH ambulance in the STH system from using CRC's sensor agents to avoid an environmental hazard.

5.2.3. Agent Network Protocol (ANP)

ANP offers a decentralized vision that addresses the security and trust challenges of the STH scenario. By leveraging W3C DIDs and Verifiable Credentials, ANP enables trustless interactions between ambulances and RSUs, granting priority access only to legitimate vehicles while maintaining high security against identity spoofing. The primary obstacle for ANP in the STH scenario is the computational and communication overheads of its P2P architecture. DID resolution and authentication in a P2P network often involve complex cryptographic operations that can take 150–200 ms, as illustrated in [64], well beyond the latency requirements of STH. In a high-speed vehicular environment, these delays could cause the ambulance to pass the RSU before the signal preemption request is fully authenticated and processed.

In the CRC scenario, ANP's decentralized identity and discovery model perfectly match the open community ethos. Researcher agents can utilize the P2P network to discover sensor agents across organizational boundaries. The semantic richness of JSON-LD ensures that data from a sensor agent in one region can be understood and used by a model-proposing agent in another, effectively bridging the heterogeneity of the open community. However, P2P signaling required for agent discovery can trigger a network-wide signaling storm if not properly managed.

During parallel operations of STH and CRC, ANP's decentralized architecture eliminates the bottleneck of a central registry. However, without a sophisticated quality-of-service (QoS) mechanism, the high-volume discovery requests from the CRC community could interfere with the time-sensitive interactions among agents in the STH system.

5.2.4. Language Model Operating System (LMOS)

In the STH scenario, LMOS provides a bridge across the infrastructure continuum. Its use of the WoT Thing Description (TD) allows ambulance agents to interact with a cloud-based hospital admission system via HTTP while simultaneously polling a local medical sensor via CoAP or MQTT. This protocol-agnosticism is vital for integrating the diverse hardware found in medical emergency environments. LMOS's support for both local (mDNS) and global (registry) discovery enables ambulances to maintain connectivity in both urban and rural contexts. However, as STH agents in vehicular networks exchange TDs to establish connections, the computational cost of metadata propagation can be significant. Studies show that JSON-LD resolution can consume up to 80% of a file's total processing time in real-time systems [65]. The overhead of TD updates as ambulances

move across different network segments can lead to "metadata lag", in which an agent attempts to communicate with an RSU that is no longer accessible.

For the CRC scenario, LMOS's protocol-agnostic nature enables agents to expose their services simultaneously via HTTP for cloud servers and via MQTT for low-power edge sensors. This ensures maximal reach within the large-scale network. The centralized registry option in LMOS is perfect for the CRC's static directory of researcher agents, while the P2P option provides a fallback if the primary registry becomes unreachable. On the other hand, LMOS's computational overheads for ontology management may degrade the CRC system's performance as the system scales.

In parallel operations of STH and CRC systems, LMOS's intelligent task management and routing functions can dynamically route queries to the most suitable agents, taking the infrastructure's current load into account. This allows LMOS to prioritize the execution of critical STH medical actions on high-performance nodes while relegating routine CRC processing to lower-priority resources.

5.2.5. AConnP and SLIM Protocols from AGNTCY

The AConP and SLIM messaging from the AGNTCY project provide high-performance information sharing in the STH scenario [66]. SLIM's gRPC-based transport uses binary serialization over HTTP/2 to achieve the low latency and high throughput required for real-time ambulance navigation and RSU signal control in this scenario. For example, quantitative analysis in [62] demonstrates an end-to-end response time of 25 ms and a throughput of 50,000 requests per second achieved by gRPC over HTTP/2. Furthermore, SLIM's binary Protobuf serialization may reduce payload size by 60% to 80% compared to JSON, as indicated in [67], which is critical in STH for minimizing bandwidth usage in vehicular networks. However, the AGNTCY framework assumes a degree of network stability that may not hold during high-speed emergency transits across heterogeneous network zones (e.g., from a 5G cellular network to a localized Wi-Fi node of a localized RSU).

In the CRC scenario, AGNTCY's ADS provides a scalable, DHT-based registry that avoids the bottlenecks of centralized agent discovery. The OASF schema supports resource-aware agent descriptions, which are critical for selecting the right analysis agent based on task complexity and agents' computational capacities. AGNTCY's SLIM is ideal for the high-bandwidth exchange of research artifacts and datasets. The efficiency of binary payloads ensures that large-scale environmental models can be transferred between researcher and actuator agents without clogging the network. However, the AGNTCY stack's cloud-centric design assumes stable network connections, which may be unrealistic for CRC sensor agents in remote areas with high packet loss.

When operating STH and CRC in parallel, the OASF framework and SLIM's stream multiplexing in AGNTCY allow the infrastructure to allocate specific bandwidth and computational resources to each MAS, preventing data sharing in CRC from interfering with the STH's critical communications [66]. However, the framework's high operational complexity may exceed the management capacity of smaller organizations participating in the CRC scenario.

5.2.6. Merged A2A and ACP Protocol

The merged A2A protocol (A2A + ACP) represents a more comprehensive attempt to meet the requirements of both the STH and CRC systems. For the STH scenario, it offers a dual-stack approach that allows an ambulance to use gRPC for video and telemetry while maintaining a RESTful interface for lower-priority administrative communications. The merged protocol also integrates the OASF, allowing agents to describe their resource constraints (e.g., GPUs, memory, batteries), which is critical in the STH scenario; for example,

it allows an ambulance agent to selectively discover a medical server with available GPU capacity for real-time diagnostic processing.

In the CRC scenario, the merged protocol's support for JSON-LD ensures semantic interoperability, while its use of OASF enables incremental updates to agent records. This means a research agent can update only a small portion of its metadata—such as its current availability or a new sensor skill—without re-transmitting its entire Agent Card, saving a significant amount of bandwidth in a network comprising numerous agents.

In parallel operation of the STH and CRC systems, integrating both OAuth and DIDs into the merged protocol provides a flexible security model that leverages high-speed OAuth for connected STH operations and decentralized DID-based trust for CRC-style cross-organizational collaboration. Its support for multiple transport and discovery schemes also enables interoperability between STH and CRC, facilitating cross-domain collaboration during an emergency. The primary limitation of the merged A2A protocol in a parallel-operation scenario is its inability to provide infrastructure-level resource isolation and contention management, mainly due to a lack of native mechanisms to coordinate with the underlying edge–network–cloud infrastructures for Quality of Service (QoS) provisioning.

6. Unified Framework for AI Agent Communications in Agentic Web

In this section, we first identify the gap between the capabilities of representative agent communication protocols and the demands of the Agentic Web. To address these gaps, we advocate a unified architectural framework for Agentic Web that embraces the virtualization and service-oriented paradigms for agent communication in the future Internet. We then discuss key topics and potential directions for future research to realize this Agentic Web framework.

6.1. Limitations of the Current Agent Communication Protocols for Agentic Web

The survey and analysis presented in the previous sections indicate that, although encouraging progress has been made in the technologies and protocols for inter-agent communication, realization of the Agentic Web in the future internet still faces two main hurdles: the *Interoperability Crisis* and the *Infrastructure Gap*.

All agent communication protocols are designed with interoperability among heterogeneous AI agents as their primary objective. However, the proliferation of agent protocol designs has led to the coexistence of competing protocols, thereby introducing interoperability issues on the communication level while addressing them at the agent level. Recent developments, exemplified by the merger of A2A and ACP protocols, reflect attempts to develop a unified protocol to address interoperability issues at the protocol level. However, the current landscape and trends in agent protocols indicate that no single protocol is likely to dominate agent communications across all agentic AI application scenarios. Our analysis reveals a more fundamental challenge to the unified protocol strategy for resolving the interoperability crisis: no single technology can be optimized to address all the conflicting requirements of agent communication across diverse scenarios. For example, the technologies that are best for efficiency and dynamism, which are critical for agent communication in edge computing environments, are often the worst for heterogeneity, a key requirement for enabling interoperability across heterogeneous agents. Therefore, agents in cloud data centers prefer high-throughput, low-latency gRPC, whereas agents on battery-powered mobile sensors tend to use lightweight, asynchronous transport protocols such as CoAP or MQTT.

The current agent protocols also aim to enhance infrastructure awareness and to provide various cloud-centric or edge-native designs. Examples of developments in this

direction include the intelligent task management and routing functions in LMOS for task allocation and load balancing, and the adoption of OSAF in both the AGNTCY project and the merged A2A protocol for resource-aware agent description and discovery. However, there is still a lack of an effective mechanism that enables smooth cooperation between resource-aware management at the agent communication layer and the control mechanisms in edge–network–cloud infrastructures, e.g., task scheduling in edge computing systems, bandwidth allocation in networks, and virtual machine scaling in cloud data centers. Therefore, the infrastructure gap of agent communication remains unfilled. A key to addressing this problem lies in cross-layer coordination that makes infrastructure status visible to agent communication protocols, while allowing upper-layer resource management to govern the control of underlying infrastructures. Any single protocol, by definition, is the interaction mechanism among modules within a single layer; therefore, it alone cannot achieve the cross-layer cooperation required to close the infrastructure gap.

6.2. Service-Oriented Virtualization-Based Framework for Agentic Web

Our analysis of current agent communication protocols implies that it would be infeasible for a single, monolithic protocol to resolve both the interoperability crisis and fill the infrastructure gap to satisfy all requirements of inter-agent communications in the vast, complex, and heterogeneous edge–network–cloud continuum of the future Internet. Therefore, we argue that agent communication for Agentic Web calls for a unified architectural framework that not only accommodates the coexistence of hybrid protocols with diverse design objectives but also adaptively leverages the appropriate protocol bundle to support the numerous MASs deployed for various agentic AI applications.

The key requirements for such a framework include the following aspects: (i) cooperability that allows hybrid agent communication protocols to operate in parallel without interference; (ii) flexibility that enables coexisting protocols to be adaptively leveraged for various MASs to meet their diverse requirements; and (iii) holisticity that supports coordination across the layers of the architecture, including agentic AI applications, multi-agent systems, the agent communication platform, and the underlying infrastructures. We believe such an architectural framework will greatly facilitate a pluralistic ecosystem in which various agent communication technologies and protocols can be freely developed and fully utilized in the Agentic Web.

Given the convergence of networking and edge/cloud computing in the future Internet, and inspired by the proven success of the virtualization paradigm and service-oriented architecture in both cloud/edge computing and future networking, we envision an Agentic Web architectural framework grounded in the virtualization and service-oriented principles.

The key notion of virtualization lies in the decoupling of a function's capabilities from the (computational and communication) resources it utilizes. Such decoupling is critical for achieving cooperability without interference by enabling diverse virtual functions to share a common infrastructure substrate, with each function using a slice of the substrate isolated from the slices used by other functions [68]. Virtualization also greatly facilitates the holistic architecture by enabling cross-layer coordination without being constrained by the implementation details of individual layers [69].

Service orientation is an architectural principle that encapsulates system modules into self-contained, platform-independent *services* and enables loosely coupled interactions among heterogeneous modules through abstract service interfaces. Service-oriented architecture provides the flexibility and interoperability required for highly integrated, cross-platform, inter-domain communication environments and has thus been widely adopted and realized through the Everything-as-a-Service (XaaS) paradigm, not only in

cloud/edge computing but also in the future Internet, e.g., via the Network-as-a-Service (NaaS) model [70].

By embracing virtualization and service-oriented paradigms for agent communication in the Agentic Web, we envision a service-oriented, virtualization-based architectural framework (SOVA), as depicted in Figure 5. This architecture comprises the Virtualized Infrastructure layer at the bottom, the MAS Communication Platform layer in the middle, and the Agentic AI Application layer at the top.

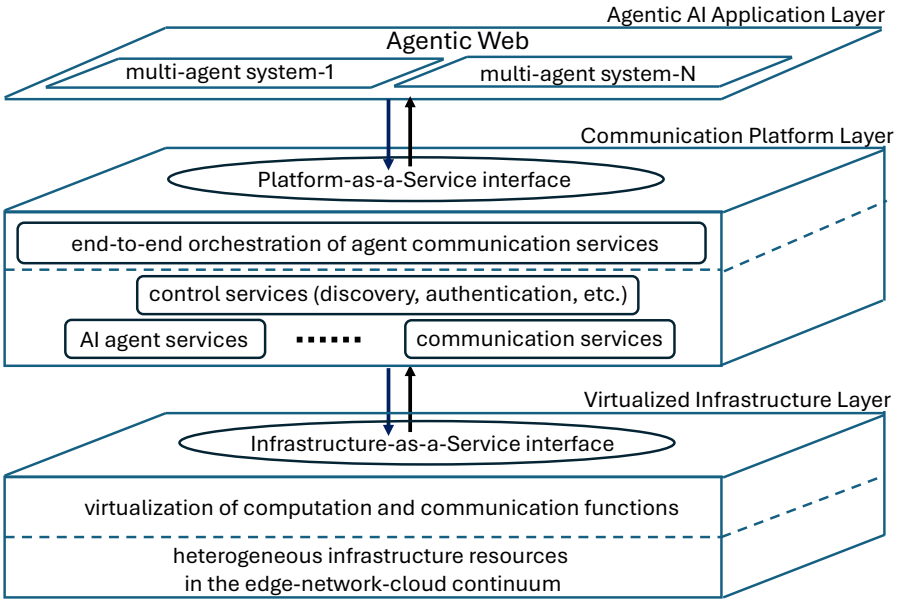

Figure 5. A service-oriented virtualization-based architectural framework for agent communications in the Agentic Web.

The Virtual Infrastructure layer comprises heterogeneous infrastructure resources that are encapsulated into virtual computation or communication functions and composed into various edge–network–cloud infrastructure services, which are then provisioned to the Platform layer via the Infrastructure-as-a-Service (IaaS) interface between the Infrastructure and Platform layers.

On the Platform layer, various AI agents and agent-communication functionalities are encapsulated as virtual services via the Agent-as-a-Service and Network-as-a-Service paradigms, each is hosted on its corresponding infrastructure service. Various multi-agent systems can be constructed by composing the required service functions, including description, discovery, and orchestration of appropriate agent communication services, into end-to-end MAS services, which are then offered to upper-layer agentic applications following the Platform-as-a-Service (PaaS) model through the interface between the Platform and Application layers.

On the top layer, various Agentic AI applications can be developed by leveraging the MAS services that the Platform layer has composed and provided to meet specific application requirements. These applications can then be delivered to end users as agentic AI services following the Application-as-a-Service (AaaS) model.

The virtualization principle, when embraced in this layered Agentic Web architecture, enables a set of virtual edge–network–cloud continuum slices, implemented using different agent communication protocols that leverage their corresponding slices of the infrastructure substrate to host various MASs. For example, in order to operate the STH and CRC systems in the illustrative Agentic Web scenario presented in Section 2.2, the Internet infrastructure can be virtualized into different network slices [71], including slices for

URLLC (Ultra Reliable and Low Latency Communications) [72], for eMBB (enhanced Mobile Broadband) [73], and for mMTC (massive Machine Type Communications) [74]. The SOVA framework can compose the URLLC and eMBB slices as the network infrastructure for the STH system, which adopts the SLIM/gRPC bundle to enable high-throughput information sharing among medical facilities and real-time traffic signal control. The framework may choose to deploy the CRC system across a combination of mMTC and eMBB slices, employing a lightweight MQTT/CoAP bundle on the mMTC slice for cost-efficient communication among the massive number of edge-based sensing agents, while leveraging SLIM/gRPC between cloud-based analyst agents to exchange how-volume model data.

The service-oriented principle may be applied in two dimensions to the Agentic Web architecture: the vertical dimension across layers and the horizontal dimension within layers. In the vertical dimension, service-orientation allows loosely coupled interactions across inter-layer interfaces by abstracting layer implementations, thereby enabling holistic cross-layer cooperation in the architecture. In the horizontal dimension, MASs can be constructed by composing and orchestrating service functions that abstract agent communication capabilities, thus significantly enhancing the architecture's flexibility. For example, the underlying network slices, e.g., URLLC, eMBB, and mMTC, can be encapsulated into virtual network services [75], and their capabilities and resource availability can be published through the service publication and discovery mechanism in the SOVA framework. Then, the service-orchestration module on the communication platform layer can discover, select, and compose the appropriate bundles of network slice services, e.g., URLLC+eMMB slices for STH and mMTC+eMBB slices for CRC, into end-to-end network infrastructure services to meet their respective requirements [76]. The network slice-as-a-service (NSaaS) paradigm also allows the orchestration module to regularly check the infrastructure status, such as bandwidth capacity and connection states, via abstract service interfaces, e.g., using the RESTful GET message, without exposing network operation details, thereby enabling the control plane on the communication platform layer to adaptively adjust its selection of agent protocols.

6.3. Possible Topics and Directions for Future Research

In this subsection, we identify some critical topics and potential directions for future research to realize the SOVA framework and enable the Agentic Web in the future Internet.

6.3.1. Flexible and Efficient Descriptions for Diverse Services

The XaaS paradigm adopted in the SOVA framework calls for further study of description approaches for the wide range of services across all layers of this framework, including network-compute infrastructure services, services that encapsulate agent control and information transport functions, and services provided by multi-agent systems. The expected service descriptions should strike a balance between rich semantics, which are required for interoperability and flexibility to address the challenges of heterogeneity and dynamicity, and simple representation, which is critical for efficiency and scalability in resource-constrained systems. A promising approach worth further exploration is the incremental metadata update principle and the hot-reloading scheme, as adopted in the OASF framework, which utilizes differential semantic updates to reflect changes in state, such as current resource availability or new network connections, rather than retransmitting entire Agent Cards or manifests.

6.3.2. Capability-Based, Trust-Weighted, and Performance-Oriented Service Discovery and Selection

Service discovery and selection mechanisms are expected to play a crucial role in the SOVA framework and warrant further research. The discovery of highly diverse services (including both agent and infrastructure services) in this framework, and the selection of an appropriate set of services to build the required MASs, should be both capability-based and performance-oriented. That is, the decision of service discovery and selection should be based not only on the functionalities provided by available services but also on the quality of service provisioning, so that the composition of selected services not only accomplishes the tasks of the multi-agent system but also meets the performance requirements of the MAS. In addition, agent discovery in the Agentic Web framework should support trust management, which cannot rely solely on agents' own descriptions and should offer additional mechanisms, such as reputation or behavior-based methods, for assessing the trustworthiness of available agents.

6.3.3. Inter-Domain and Cross-Layer Service Orchestration

Service orchestration is central to system management and control in the SOVA framework and thus presents a critical topic for future research. Two key aspects of service orchestration deserve thorough investigation. The first aspect is end-to-end cross-domain orchestration, which focuses on coordinating the diverse services across heterogeneous domains. The heterogeneity of domains may arise from either their implementations (e.g., AI agents using different LLMs or utilizing heterogeneous infrastructures) or their administration (e.g., AI agents operated by different organizations). The other key aspect of service orchestration research is cross-layer cooperation, which requires the seamless integration of infrastructure-aware and performance-oriented service management to ensure that composite agent-communication services fully leverage underlying infrastructure resources while meeting end-to-end performance requirements. The promising service orchestration architecture in the SOVA framework is a hybrid approach that enables decentralized coordination across autonomous domains while allowing each domain to choose its own orchestration mechanism.

6.3.4. Balanced Layer-Decoupling with Cross-Layer Cooperation

Another important research topic is the technical strategy for balancing layer decoupling and cross-layer cooperation in the SOVA framework. On the one hand, the virtualization principle of this framework requires decoupling higher-layer agent/communication service functions from their specific implementations in the underlying edge–network–cloud infrastructure, which is critical for enabling multiple virtual MASs to share the infrastructure substrate while utilizing diverse agent protocols tailored for different application requirements. On the other hand, infrastructure-aware, performance-oriented service orchestration requires cross-layer cooperation within the SOVA framework. Therefore, how to design cross-layer interfaces that balance these two competing demands remains an open issue for further study, and a key to solving it may lie in an optimal level of information abstraction that allows sufficient information to be exchanged between layers while maintaining the transparency of the underlying infrastructure to upper-layer functions. Cross-layer signaling that allows the platform layer to communicate its intentions, including service-quality requirements, to the underlying infrastructure is a key enabler of cross-layer cooperation. Recent progress in agentic AI empowered Intent-Based Networking (IBN) [77] offers a promising approach that enables agents to express their needs and the network to dynamically provision a slice to meet those requirements.

6.3.5. An Integral Plane for Zero-Trust Architecture

Multi-agent collaboration in Agentic Web introduces a broad range of new attacks that may compromise the security of agentic AI systems. For example, AI agents are particularly susceptible to semantic attacks, such as prompt injection, that can lead to goal manipulation or unauthorized tool invocation. A2A spoofing attacks allow a malicious agent to impersonate an orchestrator, thereby misdirecting the entire cluster of agents in an MAS. Traditional security models based on network perimeters are insufficient for AI agents that operate continuously and autonomously across organizational boundaries. To address this within the SOVA framework, security must be viewed as an integral plane spanning all three layers with a Zero-Trust Architecture (ZTA) [78]. We believe that further investigations in the following areas are necessary to construct such a ZTA within the SOVA framework. Agent identification and authentication should be further enhanced from user credentials to cryptographically verifiable agent identities that support differentiation between a human user and an AI agent acting on her behalf. Authorization needs to be extended from role/attribute-based decision to include ephemeral intention/behavior-based access control. The system should also validate agent identity, integrity, and context for every action, not just at session initiation, which involves monitoring behavioral baselines to detect deviations that might indicate goal hijacking or compromise.

7. Conclusions

In this paper, we survey the state-of-the-art agent communication protocols and technologies and assess their effectiveness in building the Agentic Web, addressing the challenges of heterogeneity, scalability, dynamicity, efficiency, and security introduced by the edge–network–cloud continuum in the future Internet. Our review demonstrates that although encouraging progress has been made in agent communication technologies, the current agent protocols are primarily designed for specific deployment environments, either cloud-centric or edge-native systems, rather than for a holistic vision across the edge–network–cloud continuum in the future Internet. Therefore, no single available protocol is sufficiently ready to ensure the inter-agent communications required for the future Agentic Web. Our analysis also indicates that it is not feasible for any single, monolithic protocol design to satisfy all requirements for inter-agent communication across the vast, dynamic, and heterogeneous edge–network–cloud continuum of the Internet, supporting the diverse AI applications in the Agentic Web. Therefore, we argue that agent communication for Agentic Web calls for a unified architectural framework that not only accommodates the coexistence of hybrid protocols with diverse design objectives but also adaptively leverages the appropriate protocol bundle to support the numerous MASs deployed for various agentic AI applications. We believe that such an architectural framework will greatly facilitate a pluralistic ecosystem in which various agent communication technologies and protocols can be freely developed and fully utilized. In this direction, we advocate a service-oriented, virtualization-based framework for the Agentic Web, and discuss key topics and potential directions for future research to realize the framework.

Author Contributions: Conceptualization, methodology, formal analysis, investigation, writing—original draft preparation, Q.D.; resources, writing—review and editing, Z.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partially funded by the Yangtze River Delta Science and Technology Innovation Community Joint Research Project (YDZX20233100004031) and the Intel Sponsored Research Agreement (Intel CG # 89533661).

Data Availability Statement: The original contributions presented in this study are included in the article.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Xi, Z.; Chen, W.; Guo, X.; He, W.; Ding, Y.; Hong, B.; Zhang, M.; Wang, J.; Jin, S.; Zhou, E.; et al. The rise and potential of large language model based agents: A survey. *Sci. China Inf. Sci.* **2025**, *68*, 121101. [CrossRef]
2. Bandi, A.; Kongari, B.; Naguru, R.; Pasnoor, S.; Vilipala, S.V. The rise of agentic AI: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet* **2025**, *17*, 404. [CrossRef]
3. Sapkota, R.; Roumeliotis, K.I.; Karkee, M. AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenge. *arXiv* **2025**, arXiv:2505.10468. [CrossRef]
4. Karim, M.M.; Khan, S.; Van, D.H.; Liu, X.; Wang, C.; Qu, Q. Transforming data annotation with AI agents: A review of architectures, reasoning, applications, and impact. *Future Internet* **2025**, *17*, 353. [CrossRef]
5. Tran, K.T.; Dao, D.; Nguyen, M.D.; Pham, Q.V.; O'Sullivan, B.; Nguyen, H.D. Multi-agent collaboration mechanisms: A survey of LLMs. *arXiv* **2025**, arXiv:2501.06322. [CrossRef]
6. Li, X.; Wang, S.; Zeng, S.; Wu, Y.; Yang, Y. A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges. *Vicinagearth* **2024**, *1*, 9. [CrossRef]
7. Yang, Y.; Peng, Q.; Wang, J.; Zhang, W. LLM-based multi-agent systems: Techniques and business perspectives. *arXiv* **2024**, arXiv:2411.14033.
8. Zhu, X.; Li, J.; Liu, Y.; Ma, C.; Wang, W. A survey on model compression for large language models. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 1556–1577. [CrossRef]
9. Cheng, H.; Zhang, M.; Shi, J.Q. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Trans. Pattern Anal. Mach. Intell.* **2024**, *46*, 10558–10578. [CrossRef]
10. Zhang, R.; Liu, G.; Liu, Y.; Zhao, C.; Wang, J.; Xu, Y.; Niyato, D.; Kang, J.; Li, Y.; Mao, S.; et al. Toward edge general intelligence with agentic AI and agentification: Concepts, technologies, and future directions. *IEEE Commun. Surv. Tutor.* **2026**, *28*, 4285–4318. [CrossRef]
11. Duan, Q.; Wang, S.; Ansari, N. Convergence of networking and cloud/edge computing: Status, challenges, and opportunities. *IEEE Netw.* **2020**, *34*, 148–155. [CrossRef]
12. Yan, B.; Zhou, Z.; Zhang, L.; Zhang, L.; Zhou, Z.; Miao, D.; Li, Z.; Li, C.; Zhang, X. Beyond self-talk: A communication-centric survey of LLM-based multi-agent systems. *arXiv* **2025**, arXiv:2502.14321.
13. Yang, Y.; Chai, H.; Song, Y.; Qi, S.; Wen, M.; Li, N.; Liao, J.; Hu, H.; Lin, J.; Chang, G.; et al. A Survey of AI Agent Protocols. *arXiv* **2025**, arXiv:2504.16736. [CrossRef]
14. Ray, P.P. A review on Agent-to-Agent protocol: Concept, state-of-the-art, challenges and future directions. *Preprints* **2025** . [CrossRef]
15. Duan, Q.; Lu, Z. Agent Communications toward Agentic AI at Edge—A Case Study of the Agent2Agent Protocol. In *Proceedings of the 11th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom 2025)*; IEEE: New York, NY, USA, 2025.
16. Ehtesham, A.; Singh, A.; Gupta, G.K.; Kumar, S. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *arXiv* **2025**, arXiv:2505.02279.
17. Kong, D.; Lin, S.; Xu, Z.; Wang, Z.; Li, M.; Li, Y.; Zhang, Y.; Sha, Z.; Li, Y.; Lin, C.; et al. A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures. *arXiv* **2025**, arXiv:2506.19676. [CrossRef]
18. Li, G.; Hammoud, H.; Itani, H.; Khizbullin, D.; Ghanem, B. CAMEL: Communicative agents for "mind" exploration of large language model society. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 51991–52008.
19. Hong, S.; Zheng, X.; Chen, J.; Cheng, Y.; Wang, J.; Zhang, C.; Wang, Z.; Yau, S.K.S.; Lin, Z.; Zhou, L.; et al. MetaGPT: Meta programming for multi-agent collaborative framework. In Proceedings of the 12th International Conference on Learning Representations (ICLR 2024), Vienna, Austria, 7–11 May 2024.
20. Grimes, S.; Breen, D.E. A multi-agent approach to binary classification using swarm intelligence. *Future Internet* **2023**, *15*, 36. [CrossRef]
21. Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Zhang, S.; Zhu, E.; Li, B.; Jiang, L.; Zhang, X.; Wang, C. AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. In Proceedings of the 1st Conference on Language Modeling, Philadelphia, PA, USA, 7–9 October 2024.
22. Venkadesh, P.; Divya, S.; Kumar, K.S. Unlocking AI creativity: A multi-agent approach with CrewAI. *J. Trends Comput. Sci. Smart Technol.* **2024**, *6*, 338–356. [CrossRef]
23. Mavroudis, V. *Comprehensive Review of LangChain*; Technical Review; The Alan Turing Institute: London, UK, 2024.
24. Lovén, L.; Farahani, R.; Murturi, I.; Sigg, S.; Dustdar, S. Agentic Edge Intelligence: A Research Agenda. In Proceedings of the 18th IEEE/ACM International Conference on Utility and Cloud Computing, Nantes, France, 1–4 December 2025; pp. 1–5.
25. Khan, R.; Sarkar, S.; Mahata, S.K.; Jose, E. Security threats in agentic AI system. *arXiv* **2024**, arXiv:2410.14728.

26. Obsidian Security Team. The 2025 AI Agent Security Landscape: Players, Trends, and Risks. 2016. Available online: https://www.obsidiansecurity.com/blog/ai-agent-market-landscape (accessed on 3 March 2026).
27. Zhu, C.; Dastani, M.; Wang, S. A survey of multi-agent deep reinforcement learning with communication. *Auton. Agents Multi-Agent Syst.* **2024**, *38*, 4. [CrossRef]
28. Hou, X.; Zhao, Y.; Wang, S.; Wang, H. Model Context Protocol (MCP): Landscape, security threats, and future research directions. *arXiv* **2025**, arXiv:2503.23278. [CrossRef]
29. Xu, S.; Hou, H.; Sun, X.; Zhang, S. Agent Network Protocol White Paper. Available online: https://w3c-cg.github.io/ai-agent-protocol/ (accessed on 3 March 2026).
30. CopilotKit. Agent-User Interaction Protocol (AG-UI). Available online: https://docs.ag-ui.com/introduction (accessed on 3 March 2026).
31. Google. Announcing the Agent2Agent Protocol (A2A). Available online: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/ (accessed on 3 March 2026).
32. Besen, S.; Gutowska, A. What Is Agent Communication Protocol (ACP)? Available online: https://www.ibm.com/think/topics/agent-communication-protocol (accessed on 3 March 2026).
33. Chang, G.; Lin, E.; Yuan, C.; Cai, R.; Chen, B.; Xie, X.; Zhang, Y. Agent Network Protocol Technical White Paper. *arXiv* **2025**, arXiv:2508.00007.
34. Eclipse. Language Model Operating System (LMOS). Available online: https://eclipse.dev/lmos/ (accessed on 3 March 2026).
35. AGNTCY/docs. AGNTCY Origins. 2025. Available online: https://docs.agntcy.org/ (accessed on 3 March 2026).
36. Marro, S.; La Malfa, E.; Wright, J.; Li, G.; Shadbolt, N.; Wooldridge, M.; Torr, P. A Scalable Communication Protocol for Networks of Large Language Models. *arXiv* **2024**, arXiv:2410.11905. [CrossRef]
37. WildcardAI. Agents.json Specification. Available online: https://github.com/wild-card-ai/agents-json (accessed on 3 March 2026).
38. IETF. RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format. 2017. Available online: https://datatracker.ietf.org/doc/html/rfc8259 (accessed on 3 March 2026).
39. WHATWG. HTML Living Standard 9.2 Server-Sent Events. 2026. Available online: https://html.spec.whatwg.org/multipage/server-sent-events.html (accessed on 3 March 2026).
40. Pautasso, C.; Zimmermann, O.; Leymann, F. Restful web services vs. "big" web services: Making the right architectural decision. In Proceedings of the 17th International Conference on World Wide Web, WWW 2008, Beijing, China, 21–25 April 2008.
41. W3C. Decentralized Identifiers (DIDs) Version 1.1. 2026. Available online: https://www.w3.org/TR/did-1.1/ (accessed on 3 March 2026).
42. Mazzocca, C.; Acar, A.; Uluagac, S.; Montanari, R.; Bellavista, P.; Conti, M. A survey on decentralized identifiers and verifiable credentials. *IEEE Commun. Surv. Tutor.* **2025**, *27*, 3641–3671. [CrossRef]
43. W3C. JSON-LD: A JSON-Based Serialization for Linked Data Version 1.1. 2022. Available online: https://www.w3.org/TR/json-ld11/ (accessed on 3 March 2026).
44. W3C. Web of Things (WoTs) Architecture Version 1.1. Available online: https://www.w3.org/TR/wot-architecture/Overview.html (accessed on 3 March 2026).
45. IETF. RFC 6762: Multicast Domain Name System (DNS). 2013. Available online: https://datatracker.ietf.org/doc/html/rfc6762 (accessed on 3 March 2026).
46. IETF. RFC 6763: DNS-Based Service Discovery. 2013. Available online: https://datatracker.ietf.org/doc/html/rfc6763 (accessed on 3 March 2026).
47. IETF. RFC 6455: The WebSocket Protocol. 2011. Available online: https://datatracker.ietf.org/doc/html/rfc6455 (accessed on 3 March 2026).
48. OASIS. Message Queuing Telemetry Transport (MQTT) Version 5.0. 2019. Available online: https://www.oasis-open.org/standard/mqtt-v5-0-os/ (accessed on 3 March 2026).
49. IETF. RFC 7252: The Constrained Application Protocol (CoAP). 2014. Available online: https://datatracker.ietf.org/doc/html/rfc7252 (accessed on 3 March 2026).
50. Muscariello, L.; Pandey, V.; Polic, R. The AGNTCY agent directory service: Architecture and implementation. *arXiv* **2025**, arXiv:2509.18787. [CrossRef]
51. OCI. Open Container Initiative Runtime Specification. 2025. Available online: https://specs.opencontainers.org/runtime-spec/?v=v1.0.2 (accessed on 3 March 2026).
52. OpenAPI. Specification (Version 3.1.0). 2021. Available online: https://spec.openapis.org/oas/v3.1.0.html (accessed on 3 March 2026).
53. W3C. Verifiable Credentials Data Model Verson 2.0. 2025. Available online: https://www.w3.org/TR/vc-data-model-2.0/ (accessed on 3 March 2026).

54. Jankovics, V. AI Agent Protocols: ACP and A2A Unite. Available online: https://dotsquarelab.com/resources/acp-and-a2a-united (accessed on 3 March 2026).
55. LFAI&Data. ACP Joins Forces with A2A. Available online: https://lfaidata.foundation/communityblog/2025/08/29/acp-joins-forces-with-a2a-under-the-linux-foundations-lf-ai-data/ (accessed on 3 March 2026).
56. CZ. Complete Guide: Agent2Agent (A2A) Protocol—The New Standard for AI Agent Collaboration. Available online: https://dev.to/czmilo/2025-complete-guide-agent2agent-a2a-protocol-the-new-standard-for-ai-agent-collaboration-1pph (accessed on 3 March 2026).
57. Agbabian, P. *Understanding the Open Cybersecurity Schema Framework*; Technical Report; Linux Foundation: Wilmington, DE, USA, 2022.
58. Mohaisen, A.; Kim, J. The Sybil Attacks and Defenses: A Survey. *Smart Comput. Rev.* **2013**, *3*, 480–489. [CrossRef]
59. IETF. RFC 6749: The OAuth 2.0 Authorization Framework. 2012. Available online: https://datatracker.ietf.org/doc/html/rfc6749 (accessed on 3 March 2026).
60. Huh, D.; Mohapatra, P. Grounding natural language for multi-agent decision-making with multi-agentic LLMs. *arXiv* **2025**, arXiv:2508.07466.
61. Qayyum, A.; Albaseer, A.; Qadir, J.; Al-Fuqaha, A.; Abdallah, M. LLM-driven multi-agent architectures for intelligent self-organizing networks. *IEEE Netw.* 2025, *early access*. [CrossRef]
62. Dieu, L.C. Building AI-Powered APIs: REST vs. GraphQL vs. gRPC for Real-Time ML Applications. 2025. Available online: https://smartdev.com/ai-powered-apis-grpc-vs-rest-vs-graphql/ (accessed on 3 March 2026).
63. HAFDI, A. JSON vs TOON: Choosing the Right Data Format for the AI Era. 2025. Available online: https://medium.com/@ahmed.hafdi.contact/json-vs-toon-choosing-the-right-data-format-for-the-ai-era-1a61aaa56dec (accessed on 3 March 2026).
64. Bappy, F.H.; Hossain, T.; Hasan, R.; Hasan, K.; Younis, M.; Islam, T. SWORD: A secure Low-latency offline-first authentication and data sharing scheme for resource-constrained distributed networks. *arXiv* **2026**, arXiv:2601.12875.
65. Talluri, S.; Di Donato, G.W.; Danelutti, L.; Vlaswinkel, K.R.; Arnaboldi, M.; Delamare, A.; Santambrogio, M.D.; Bonetta, D. GpJSON: High-performance JSON data processing on GPUs. *Proc. VLDB Endow.* **2025**, *18*, 3216–3229. [CrossRef]
66. Muscariello, L.; Papalini, M.; Sardara, M.; Betts, S. An Overview of Messaging Systems and Their Applicability to Agentic AI. 2025. Available online: https://www.ietf.org/archive/id/draft-mpsb-agntcy-messaging-00.html (accessed on 3 March 2026).
67. Shatnawi, A.; Bahri, A.; Niang, B.; Verhaeghe, B. Enhancing data serialization efficiency in REST services: Migrating from JSON to protocol buffers. In *Proceedings of the 20th International Conference on Software Technologies*; SCITEPRESS-Science and Technology Publications: Setúbal, Portugal, 2025; pp. 193–200.
68. Zhang, S. An overview of network slicing for 5G. *IEEE Wirel. Commun.* **2019**, *26*, 111–117. [CrossRef]
69. Duan, Q.; Ansari, N.; Toy, M. Software-defined network virtualization: An architectural framework for integrating SDN and NFV for service provisioning in future networks. *IEEE Netw.* **2016**, *30*, 10–16. [CrossRef]
70. Duan, Q. Network-as-a-service in software-defined networks for end-to-end QoS provisioning. In Proceedings of the 23rd Wireless and Optical Communication Conference (WOCC), Newark, NJ, USA, 9–10 May 2014; pp. 1–5.
71. Rafique, W.; Barai, J.R.; Fapojuwo, A.O.; Krishnamurthy, D. A survey on beyond 5G network slicing for smart cities applications. *IEEE Commun. Surv. Tutor.* **2024**, *27*, 595–628. [CrossRef]
72. Haque, M.E.; Tariq, F.; Khandaker, M.R.; Hossain, M.S.; Imran, M.A.; Wong, K.K. A comprehensive survey of 5G URLLC and challenges in the 6G era. *arXiv* **2025**, arXiv:2508.20205. [CrossRef]
73. Taskou, S.K.; Rasti, M.; Hossain, E. End-to-end resource slicing for coexistence of eMBB and URLLC services in 5G-advanced/6G networks. *IEEE Trans. Mob. Comput.* **2023**, *23*, 8015–8032. [CrossRef]
74. Yang, B.; Wei, F.; She, X.; Jiang, Z.; Zhu, J.; Chen, P.; Wang, J. Intelligent random access for massive-machine type communications in sliced mobile networks. *Electronics* **2023**, *12*, 329. [CrossRef]
75. Grings, F.H.; Bruno, G.Z.; Prade, L.R.; Both, C.B.; Brito, J.M.C. NASP: Network slice as a service platform for 5G networks. *arXiv* **2025**, arXiv:2505.24051. [CrossRef]
76. Dalgitsis, M.; Cadenelli, N.; Serrano, M.A.; Bartzoudis, N.; Alonso, L.; Antonopoulos, A. Cloud-native orchestration framework for network slice federation across administrative domains in 5G/6G mobile networks. *IEEE Trans. Veh. Technol.* **2024**, *73*, 9306–9319. [CrossRef]
77. Jiang, G.; Wang, K.; Chen, X.; Huang, Y. Agentic AI Empowered Intent-Based Networking for 6G. *arXiv* **2026**, arXiv:2601.06640. [CrossRef]
78. Campbell, R. Zero Trust for AI Systems: A Reference Architecture and Assurance Framework. *Preprints* **2026**. [CrossRef]